École Supérieure Privée d'Ingénierie et de Technologies

Honoris United Universities

RAPPORT DE PROJET

Module : IA for Software Engineering

Zitouna AI

Plateforme d'intelligence artificielle au service des petits oléiculteurs tunisiens

Réalisé par

Omar Azouz	M1 · Maladies des feuilles
Khadija Ben Ayed	M2 · Irrigation et alertes météo
Med Amine Bejaoui	M3 · Prévision de la récolte
Emna Dorai	M4 · Prix et moment de vente
Rima Jenhani	M5 · Assistant en derja
Aziz Dlissi	M6 · Comptage des oliviers

Encadré par : Khemiri Akram

Classe : 5SAE10

Année universitaire 2026 – 2027

Résumé

Le petit oléiculteur tunisien prend souvent ses décisions sans outil : il reconnaît mal les maladies, arrose « à l'œil », ignore sa récolte à venir et vend son huile sans savoir si le prix va monter. Zitouna AI est une application web qui l'accompagne grâce à six modules d'intelligence artificielle : diagnostic des maladies des feuilles par photo (MobileNetV3), conseil d'irrigation et alertes météo (LSTM), prévision de la récolte (Random Forest), prévision du prix de l'huile et conseil de vente (SARIMA), assistant conversationnel en dialecte tunisien (classification TF-IDF et grand modèle de langage) et comptage des oliviers sur image aérienne (OpenCV).

Chaque modèle est entraîné sur des données réelles et publiques, puis comparé à une méthode de référence simple sur des données de test jamais vues : tous font mieux que leur référence. Les modules sont intégrés dans une architecture complète : frontend Angular, backend Spring Boot sécurisé par JWT, base PostgreSQL, six microservices FastAPI, le tout déployé avec Docker Compose et vérifié par 154 tests automatiques.

Mots-clés : intelligence artificielle, oléiculture, vision par ordinateur, séries temporelles, traitement du langage, Spring Boot, Angular, FastAPI, Docker.

Abstract

Small olive growers in Tunisia often make decisions without any tool: diseases are hard to recognise, irrigation is guessed, the coming harvest is unknown and olive oil is sold without knowing whether prices will rise. Zitouna AI is a web application that supports them with six artificial intelligence modules: leaf disease diagnosis from a photo (MobileNetV3), irrigation advice and weather alerts (LSTM), harvest forecasting (Random Forest), olive oil price forecasting with selling advice (SARIMA), a chatbot in Tunisian Arabic (TF-IDF classification and a large language model) and olive tree counting on aerial images (OpenCV).

Each model is trained on real, public data and compared with a simple baseline on unseen test data: all of them beat their baseline. The modules are integrated into a complete architecture: an Angular frontend, a Spring Boot backend secured with JWT, a PostgreSQL database and six FastAPI microservices, deployed with Docker Compose and checked by 154 automated tests.

Keywords: artificial intelligence, olive growing, computer vision, time series, natural language processing, Spring Boot, Angular, FastAPI, Docker.

Table des matières

Introduction générale

Chapitre 1 : Contexte général du projet

1.1 Introduction
1.2 Cadre du projet
1.3 Problématique
1.4 Objectifs du projet
1.5 Contribution aux objectifs de développement durable
1.6 Organisation de l'équipe
1.7 Méthodologie
1.8 Conclusion

Chapitre 2 : Analyse et spécification des besoins

2.1 Introduction
2.2 Identification des acteurs
2.3 Besoins fonctionnels
2.4 Besoins non fonctionnels
2.5 Diagramme de cas d'utilisation
2.6 Conclusion

Chapitre 3 : Conception

3.1 Introduction
3.2 Architecture générale
3.3 Déroulement d'une prédiction
3.4 Base de données
3.5 API du backend
3.6 Choix technologiques
3.7 Conclusion

Chapitre 4 : Réalisation des modules d'intelligence artificielle

4.1 Introduction
4.2 Données utilisées
4.3 M1 · Diagnostic des maladies des feuilles
4.4 M2 · Irrigation et alertes météo
4.5 M3 · Prévision de la récolte
4.6 M4 · Prix de l'huile et moment de vente
4.7 M5 · Assistant conversationnel en derja
4.8 M6 · Comptage des oliviers
4.9 Synthèse des résultats
4.10 Conclusion

Chapitre 5 : Réalisation de l'application, tests et déploiement

5.1 Introduction
5.2 Environnement de travail
5.3 Présentation de l'application
5.4 Captures de l'application
5.5 Sécurité
5.6 Tests et qualité
5.7 Captures des tests
5.8 Déploiement
5.9 Conclusion

Chapitre 6 : IA générative, qualité et sécurité du logiciel

6.1 Introduction
6.2 Outils d'IA générative utilisés
6.3 Démarche et exemples de prompts
6.4 Gains de productivité observés
6.5 Détection de bugs et tests assistés par l'IA
6.6 Revue de sécurité
6.7 Recommandations de refactoring
6.8 Adaptation et optimisation des modèles
6.9 Limites éthiques et techniques
6.10 Conclusion

Conclusion générale et perspectives

Bibliographie et webographie

Annexes

Annexe A : Lancer l'application
Annexe B : Glossaire
Annexe C : Documentation individuelle

Liste des figures

Figure 1 : Diagramme de cas d'utilisation global
Figure 2 : Architecture générale de Zitouna AI
Figure 3 : Diagramme de séquence du diagnostic d'une feuille
Figure 4 : Modèle de la base de données
Figure 5 : Matrice de confusion de MobileNetV3 sur les photos de test
Figure 6 : Cartes Grad-CAM : en rouge, la zone utilisée pour décider
Figure 7 : Exploration des données : ET_0 mensuelle par gouvernorat et pluie annuelle
Figure 8 : Erreur de prévision de l'ET_0 selon l'horizon
Figure 9 : Production prévue un an à l'avance, comparée à la production réelle
Figure 10 : Importance des variables du Random Forest
Figure 11 : Prix de l'huile d'olive en TND/kg depuis 1992
Figure 12 : Prévisions à 3 mois sur la période de test, avec l'intervalle à 80 %
Figure 13 : Répartition des 500 questions par intention
Figure 14 : Matrice de confusion des intentions sur 20 % de questions mises de côté
Figure 15 : Les étapes du détecteur sur un verger de test aux couronnes collées
Figure 16 : Arbres réels et arbres détectés sur six vergers de test
Figure 17 : Page d'état du backend Spring Boot : base de données et six modèles réels en ligne
Figure 18 : Écran de connexion commun aux agriculteurs et à l'administrateur
Figure 19 : Accueil de l'agriculteur : résumé de l'exploitation et accès aux six outils d'IA
Figure 20 : Liste des parcelles et formulaire d'ajout
Figure 21 : M2 · Conseil d'irrigation sur 7 jours pour la parcelle « Verger de Sfax »
Figure 22 : M3 · Récolte estimée (997 kg), M6 · 86 arbres détectés et plan de récolte M6 → M3 → M4
Figure 23 : M1 · Diagnostic d'une feuille : œil de paon détecté et carte Grad-CAM
Figure 24 : M4 · Prévision du prix de l'huile et conseil de vente
Figure 25 : M5 · Questions en derja et réponses de l'assistant rédigées par Gemini
Figure 26 : Historique des prédictions de l'agriculteur
Figure 27 : Back-office : tableau de bord de l'administrateur et état des six modèles
Figure 28 : Back-office : gestion des comptes utilisateurs
Figure 29 : Back-office : détail de toutes les prédictions IA
Figure 30 : Tests du backend Spring Boot (Maven, JUnit 5) : 44 tests réussis
Figure 31 : Tests du frontend Angular (Vitest) : 15 tests réussis
Figure 32 : Tests du service M1 · Maladies (pytest) : 7 tests réussis
Figure 33 : Tests du service M2 · Irrigation (pytest) : 8 tests réussis
Figure 34 : Tests du service M3 · Récolte (pytest) : 7 tests réussis
Figure 35 : Tests du service M4 · Prix (pytest) : 9 tests réussis
Figure 36 : Tests du service M5 · Assistant (pytest) : 55 tests réussis, 1 test en direct ignoré
Figure 37 : Tests du service M6 · Comptage (pytest) : 9 tests réussis
Figure 38 : Analyse Bandit avant correction : deux alertes de gravité moyenne
Figure 39 : Bandit après correction et npm audit : aucun problème

Liste des tableaux

Tableau 1 : Contribution de Zitouna AI aux objectifs de développement durable
Tableau 2 : Répartition des modules et des rôles
Tableau 3 : Besoins fonctionnels de l'agriculteur
Tableau 4 : Besoins fonctionnels de l'administrateur
Tableau 5 : Besoins non fonctionnels
Tableau 6 : Principales routes de l'API du backend

Tableau 7 : Choix technologiques
Tableau 8 : Données utilisées par les six modules
Tableau 9 : Résultats du module M1
Tableau 10 : Résultats du module M2
Tableau 11 : Résultats du module M3
Tableau 12 : Résultats du module M4
Tableau 13 : Résultats du module M5
Tableau 14 : Résultats du module M6
Tableau 15 : Synthèse : chaque modèle servi comparé à sa référence
Tableau 16 : Pages de l'application
Tableau 17 : Tests automatiques par partie
Tableau 18 : Outils d'IA générative et outils de contrôle
Tableau 19 : Exemples de prompts et de leur vérification
Tableau 20 : Gains et limites de l'IA générative dans le développement
Tableau 21 : Défauts détectés avec l'assistance de l'IA
Tableau 22 : Recommandations de refactoring issues de la revue
Tableau 23 : Techniques d'adaptation et d'optimisation appliquées
Tableau 24 : Limites et perspectives
Tableau 25 : Modèle de fiche de documentation individuelle
Tableau 26 : Fiche de documentation individuelle de Khadija Ben Ayed

Liste des abréviations

Abréviation	Signification
API	Application Programming Interface : interface d'échange entre deux programmes
CNN	Convolutional Neural Network : réseau de neurones convolutif
ERA5	Réanalyse météorologique du centre européen ECMWF
ET_0	Évapotranspiration de référence (mm par jour)
FAO	Organisation des Nations unies pour l'alimentation et l'agriculture
FMI	Fonds monétaire international
JWT	JSON Web Token : jeton signé qui prouve l'identité de l'utilisateur
LLM	Large Language Model : grand modèle de langage
LSTM	Long Short-Term Memory : réseau de neurones récurrent
MAE	Mean Absolute Error : erreur absolue moyenne
MAPE	Mean Absolute Percentage Error : erreur moyenne en pourcentage
ODD	Objectifs de développement durable des Nations unies
ONAGRI	Observatoire national de l'agriculture (Tunisie)
REST	Representational State Transfer : style d'API web
SARIMA	Seasonal AutoRegressive Integrated Moving Average
TF-IDF	Term Frequency – Inverse Document Frequency
TND	Dinar tunisien

Introduction générale

L'olivier occupe une place à part dans l'agriculture tunisienne : il couvre une grande partie des terres cultivées, fait vivre de nombreuses familles rurales et place le pays parmi les premiers exportateurs mondiaux d'huile d'olive. Pourtant, la plupart des exploitations sont petites et leurs propriétaires disposent de peu d'outils pour décider : traiter ou non une maladie, combien arroser, quelle récolte attendre, quand vendre l'huile.

Dans le même temps, l'intelligence artificielle a beaucoup progressé : un téléphone suffit aujourd'hui pour reconnaître une image, des modèles de séries temporelles prévoient la météo ou les prix, et les modèles de langage comprennent des questions écrites librement. Ces progrès restent pourtant rarement accessibles au petit agriculteur, dans sa langue et pour ses besoins concrets.

C'est dans ce cadre que s'inscrit notre projet, réalisé dans le module *IA for Software Engineering* à ESPRIT : concevoir et réaliser **Zitouna AI**, une application web qui réunit six modules d'intelligence artificielle, chacun porté par un membre de l'équipe, et qui répond à une question simple : comment aider le petit oléiculteur à prendre de meilleures décisions, chiffres à l'appui ?

Ce rapport est organisé en six chapitres. Le premier présente le contexte, la problématique, les objectifs et les objectifs de développement durable visés. Le deuxième analyse les besoins fonctionnels et non fonctionnels. Le troisième décrit la conception de la solution. Le quatrième détaille, avec la même structure pour chaque membre, la réalisation des six modules d'intelligence artificielle. Le cinquième présente l'application réalisée, les tests et le déploiement. Le sixième décrit l'usage de l'IA générative dans le développement : prompts, détection de bugs, revue de sécurité, optimisation des modèles et limites éthiques. Nous terminons par une conclusion générale et des perspectives.

Chapitre 1 : Contexte général du projet

1.1 Introduction

Ce chapitre situe le projet : le cadre dans lequel il a été réalisé, le problème auquel il répond, les objectifs fixés, la contribution du projet aux objectifs de développement durable, l'organisation de l'équipe et la méthodologie suivie.

1.2 Cadre du projet

Le projet a été réalisé par une équipe de six étudiants dans le cadre du module *IA for Software Engineering* à l'École Supérieure Privée d'Ingénierie et de Technologies (ESPRIT). L'objectif pédagogique est double : appliquer des méthodes d'apprentissage automatique à un problème réel, et les intégrer dans une application logicielle complète, testée et déployable.

1.3 Problématique

L'huile d'olive est un produit stratégique pour la Tunisie : le pays compte 1 625 huileries, dont 931 en activité en janvier 2023 selon l'ONAGRI, avec un taux d'extraction qui varie de 13 % à 31 % selon les régions. Les prix au producteur sont très volatils et descendent parfois sous le coût de production.

Le petit oléiculteur fait face à cinq difficultés concrètes :

- **Santé des arbres** : les maladies des feuilles, comme l'œil de paon, sont souvent reconnues trop tard.
- **Eau** : l'irrigation se fait sans mesure du besoin réel, alors que l'eau est rare et que les canicules et le gel menacent la récolte.
- **Récolte** : la production varie fortement d'une année à l'autre (phénomène d'alternance), ce qui rend la saison difficile à préparer.
- **Vente** : sans prévision du prix, l'agriculteur vend souvent au mauvais moment.
- **Accès à l'information** : les conseils techniques sont rarement disponibles dans sa langue, le dialecte tunisien (derja).

La question qui guide le projet est donc la suivante : **comment mettre à la portée du petit oléiculteur des conseils fiables et chiffrés, issus de l'intelligence artificielle, dans une application simple à utiliser ?**

1.4 Objectifs du projet

Le plan de projet fixe six objectifs, un par module :

1. Identifier la maladie d'une feuille à partir d'une photo et proposer un conseil (M1).
2. Donner chaque jour un conseil d'irrigation et des alertes gel ou canicule pour la parcelle (M2).
3. Estimer la récolte de la saison et celle de la parcelle (M3).
4. Prévoir le prix de l'huile et conseiller « vendre maintenant » ou « stocker » (M4).
5. Répondre aux questions écrites en derja (M5).

6. Compter les oliviers sur une image vue du ciel (M6).

Critère de réussite : les six modules fonctionnent dans l'application pendant la démonstration, chacun avec une métrique mesurée sur des données de test et comparée à une méthode de référence simple.

1.5 Contribution aux objectifs de développement durable

Les objectifs de développement durable (ODD) sont les 17 objectifs adoptés par les Nations unies en 2015 pour l'horizon 2030. Zitouna AI contribue directement à six d'entre eux, comme le montre le tableau suivant.

ODD	Cible visée	Contribution de Zitouna AI	Modules
ODD 2 · Faim « zéro »	2.3 Productivité et revenus des petits producteurs ; 2.4 agriculture durable et résiliente	Aide le petit oléiculteur à protéger ses arbres, à prévoir sa récolte et à mieux la valoriser.	M1, M3, M6
ODD 6 · Eau propre et assainissement	6.4 Utilisation plus efficace de l'eau	Calcule le besoin en eau réel de la parcelle à partir de la météo, pour arroser juste ce qu'il faut.	M2
ODD 8 · Travail décent et croissance économique	8.2 Productivité par l'innovation technologique	Le conseil de vente, avec une marge de prudence, améliore le prix obtenu par l'agriculteur.	M4
ODD 9 · Industrie, innovation et infrastructure	9.c Accès aux technologies de l'information	Met des outils d'IA dans les mains d'agriculteurs, dans leur langue, depuis un simple navigateur.	M5, toute l'application
ODD 12 · Consommation et production responsables	12.4 Gestion raisonnée des produits chimiques	Un diagnostic précis et précoce évite de traiter à l'aveugle et limite l'usage de produits phytosanitaires.	M1
ODD 13 · Lutte contre les changements climatiques	13.1 Résilience face aux aléas climatiques	Alertes gel et canicule, et prise en compte de la sécheresse dans les prévisions de récolte.	M2, M3

Tableau 1 : Contribution de Zitouna AI aux objectifs de développement durable

1.6 Organisation de l'équipe

Pour une répartition équitable du travail, chaque membre porte un module d'intelligence artificielle de bout en bout (données, modèle, service, intégration, écran) et assure en plus un rôle transversal pour l'ensemble du projet.

Membre	Module d'IA	Type d'IA	Rôle transversal
Omar Azouz	M1 · Maladies des feuilles	Vision : classification	Pipeline d'images partagé avec M6
Khadija Ben Ayed	M2 · Irrigation et alertes météo	Séries temporelles	Backend et intégration des API
Med Amine Bejaoui	M3 · Prévision de la récolte	Régression tabulaire	Suivi des expériences, dépôt Git

Emna Dorai	M4 · Prix et moment de vente	Séries temporelles	Frontend
Rima Jenhani	M5 · Assistant en derja	Langage : classification + LLM	Rapport, tests, démonstration
Aziz Dlissi	M6 · Comptage des oliviers	Vision : détection	Docker et déploiement

Tableau 2 : Répartition des modules et des rôles

1.7 Méthodologie

Nous avons suivi une démarche inspirée de CRISP-DM, la méthode de référence des projets de science des données, adaptée à un projet logiciel complet. Les six modules ont suivi le même cycle :

1. **Compréhension du besoin** : quelle décision de l'agriculteur le module doit-il éclairer ?
2. **Collecte et exploration des données** réelles et publiques ; les données brutes ne sont jamais modifiées.
3. **Référence simple** d'abord, pour savoir ce qu'un vrai modèle doit battre.
4. **Modélisation** : entraînement et comparaison de plusieurs modèles sur un découpage sans fuite (groupé par feuille d'origine pour les images, chronologique pour les séries temporelles).
5. **Évaluation** : choix du modèle et des réglages sur la validation, mesure finale une seule fois sur le test.
6. **Déploiement** : service FastAPI avec un contrat d'API fixe et ses tests, intégration dans le backend et le frontend, conteneur Docker.

Le travail a été organisé en itérations courtes, avec un dépôt Git unique (dossiers `notebooks/`, `ai-services/`, `backend/`, `frontend/`) et un contrat d'API commun décrit dans `docs/api-contract.md`, qui permettait à chacun d'avancer sur son module sans bloquer les autres.

1.8 Conclusion

Ce chapitre a présenté le cadre du projet, le problème du petit oléiculteur, les six objectifs fixés et leur lien avec les objectifs de développement durable. Le chapitre suivant traduit ces objectifs en besoins précis.

Chapitre 2 : Analyse et spécification des besoins

2.1 Introduction

Ce chapitre identifie les acteurs du système, puis décrit ses besoins fonctionnels (ce que l'application doit faire) et non fonctionnels (les qualités qu'elle doit avoir). Il se termine par le diagramme de cas d'utilisation.

2.2 Identification des acteurs

- **Agriculteur** : utilisateur principal. Il crée son compte, enregistre ses parcelles et utilise les six outils d'IA.
- **Administrateur** : compte unique, défini dans la configuration. Il supervise la plateforme depuis un back-office.
- **Services externes** (acteurs secondaires) : l'API Open-Meteo, qui fournit les prévisions météo, et les modèles de langage Gemini et Groq, qui rédigent les réponses de l'assistant.

2.3 Besoins fonctionnels

2.3.1 Besoins de l'agriculteur

Réf.	Besoin	Description
BF1	S'inscrire et se connecter	Créer un compte (nom, e-mail, mot de passe), puis se connecter pour accéder à son espace.
BF2	Gérer ses parcelles	Ajouter, modifier et supprimer une parcelle : nom, gouvernorat, position GPS, surface, nombre d'arbres, variété, irrigation.
BF3	Diagnostiquer une feuille (M1)	Envoyer la photo d'une feuille et recevoir la maladie détectée, la confiance, la zone utilisée pour décider et un conseil ; être prévenu si la photo n'est pas assez claire.
BF4	Consulter l'irrigation (M2)	Obtenir pour une parcelle le besoin en eau des prochains jours, en litres par arbre, et les alertes gel ou canicule.
BF5	Estimer la récolte (M3)	Obtenir la production attendue de la saison et celle de sa parcelle, avec une fourchette.
BF6	Consulter le prix (M4)	Voir la prévision du prix de l'huile sur plusieurs mois et le conseil « vendre maintenant » ou « stocker ».
BF7	Poser une question (M5)	Écrire une question en derja, en arabe ou en français, et recevoir une réponse en derja fondée sur les vrais chiffres des modules.
BF8	Compter les oliviers (M6)	Envoyer une image vue du ciel et obtenir le nombre d'arbres avec l'image annotée.
BF9	Générer un plan de récolte	Enchaîner automatiquement le comptage (M6), l'estimation de la récolte (M3) et le conseil de vente (M4).
BF10	Consulter son historique	Retrouver toutes ses prédictions passées, enregistrées en base.

Tableau 3 : Besoins fonctionnels de l'agriculteur

2.3.2 Besoins de l'administrateur

Réf.	Besoin	Description
BF11	Se connecter au back-office	Le même écran de connexion mène l'administrateur au back-office et l'agriculteur à son espace.
BF12	Consulter le tableau de bord	Voir le nombre d'utilisateurs, de parcelles et d'oliviers, l'activité sur 30 jours et l'état des six modèles.
BF13	Gérer les utilisateurs	Rechercher et supprimer des comptes agriculteurs ; le compte administrateur ne peut pas être supprimé.
BF14	Gérer les parcelles	Consulter et supprimer les parcelles de la plateforme.
BF15	Consulter les prédictions	Voir le détail de tous les appels aux modèles d'IA.

Tableau 4 : Besoins fonctionnels de l'administrateur

2.4 Besoins non fonctionnels

Réf.	Qualité	Exigence et réponse apportée
BNF1	Fiabilité des modèles	Chaque modèle doit battre une référence simple sur des données de test jamais vues ; un modèle peu sûr de lui doit le dire (seuil de confiance de M1, question de clarification de M5).
BNF2	Sécurité	Mots de passe hachés (BCrypt), authentification par jeton JWT valable 24 heures, routes d'administration réservées au rôle ADMIN, secrets hors du code.
BNF3	Confidentialité	Aucune donnée personnelle (nom, e-mail) n'est envoyée au modèle de langage ; les modèles sont entraînés sur des données publiques uniquement.
BNF4	Ergonomie	Interface claire et organisée par rubriques, formats français (dates, nombres), réponses de l'assistant dans la langue de l'agriculteur.
BNF5	Explicabilité	Les résultats sont accompagnés d'une explication : carte Grad-CAM pour M1, fourchette pour M3, intervalle de confiance pour M4, image annotée pour M6.
BNF6	Disponibilité	Si un service externe tombe (météo, LLM), l'application continue de répondre : Groq prend le relais de Gemini, puis des phrases modèles.
BNF7	Maintenabilité	Un microservice par module avec un contrat d'API fixe : changer un modèle ne touche qu'un conteneur ; code testé automatiquement.
BNF8	Portabilité	Toute la plateforme se lance sur un seul PC avec une commande Docker Compose.
BNF9	Performance	Modèles légers exécutables sans carte graphique (MobileNetV3, LSTM en NumPy, OpenCV) ; photos limitées à 10 Mo.
BNF10	Reproductibilité	Notebooks exécutables de haut en bas avec une graine fixe (42) ; données brutes jamais modifiées.

Tableau 5 : Besoins non fonctionnels

2.5 Diagramme de cas d'utilisation

La figure suivante résume les interactions des deux acteurs avec le système. Tous les cas d'utilisation, sauf l'inscription, exigent une authentification préalable.

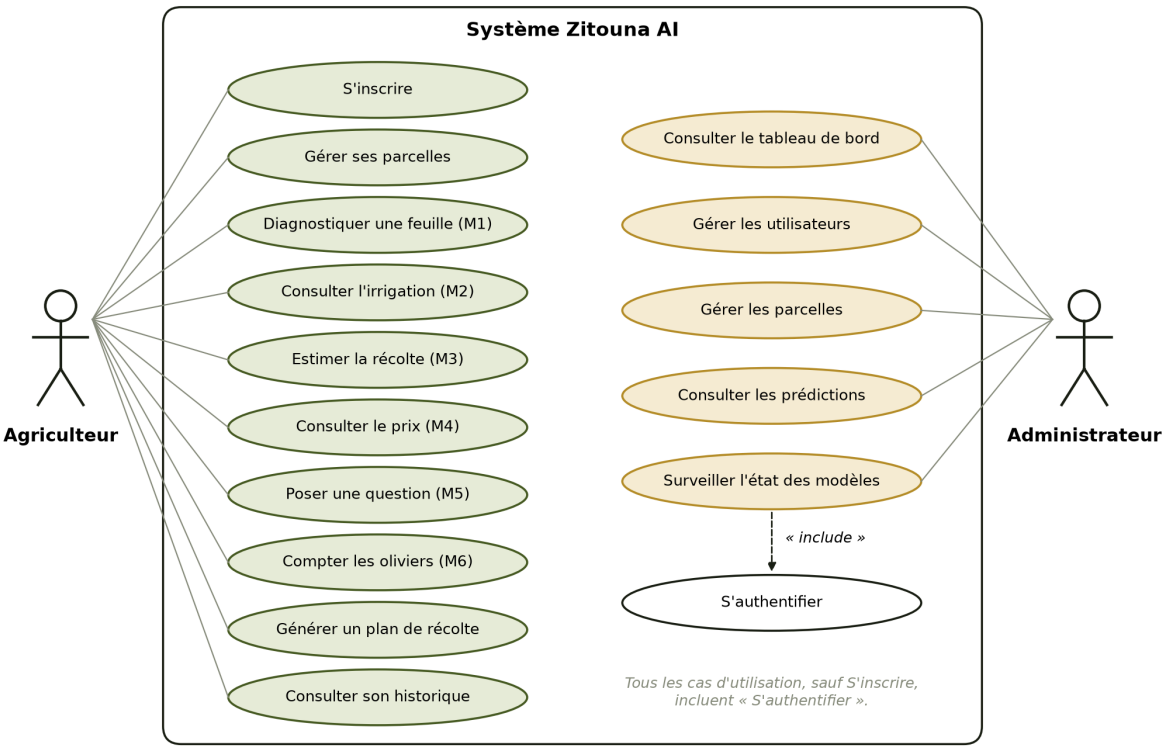


Figure 1 : Diagramme de cas d'utilisation global

2.6 Conclusion

Les besoins sont maintenant définis : quinze besoins fonctionnels répartis entre deux acteurs et dix besoins non fonctionnels. Le chapitre suivant présente la conception qui permet d'y répondre.

Chapitre 3 : Conception

3.1 Introduction

Ce chapitre présente l'architecture générale de Zitouna AI, le déroulement d'une prédiction, la base de données, l'API du backend et les choix technologiques.

3.2 Architecture générale

Nous avons choisi une architecture en microservices. L'application web (frontend) ne parle qu'au backend ; seul le backend lit la base de données et appelle les six services d'IA, chacun isolé dans son propre conteneur Docker. Ce découpage présente trois avantages : chaque membre développe et teste son module indépendamment ; changer un modèle ne touche qu'un conteneur, puisque le contrat d'API reste le même ; et la sécurité est centralisée dans le backend.

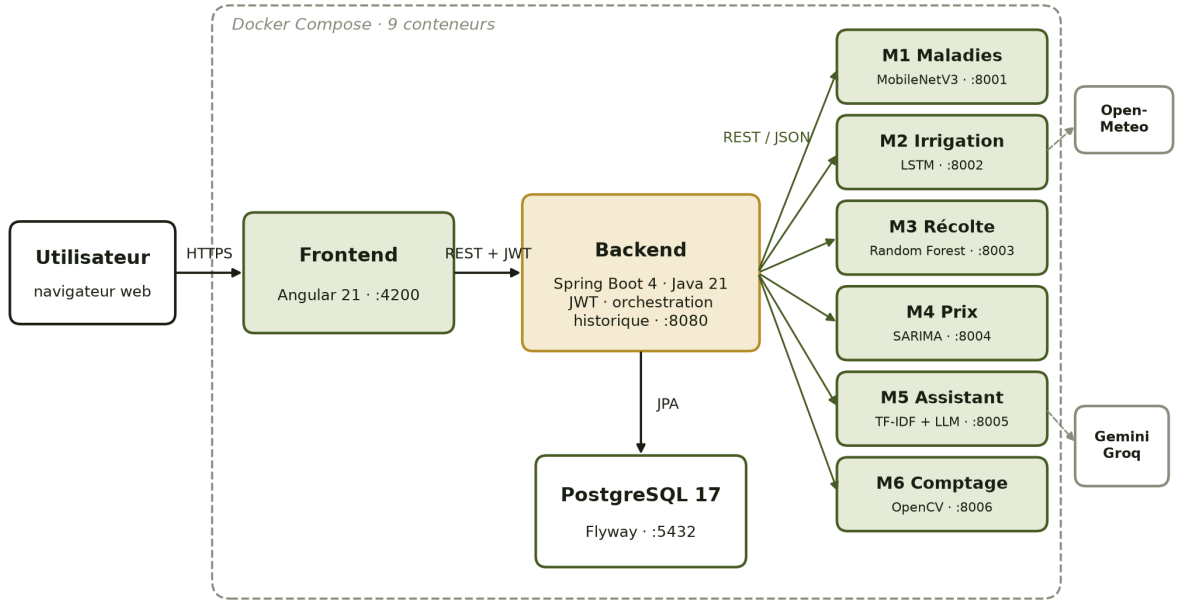


Figure 2 : Architecture générale de Zitouna AI

3.3 Déroulement d'une prédiction

Toutes les prédictions suivent le même chemin. La figure suivante l'illustre avec le diagnostic d'une feuille (M1) : le backend vérifie le jeton de l'utilisateur, appelle le service concerné, enregistre la prédiction dans l'historique, puis renvoie le résultat au frontend.

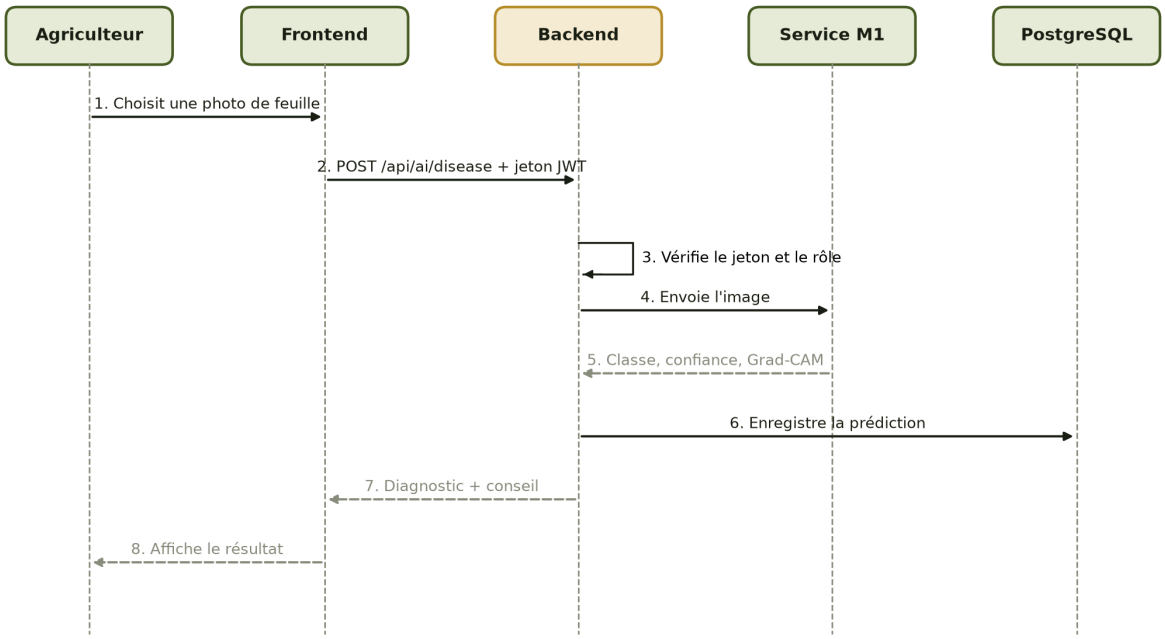


Figure 3 : Diagramme de séquence du diagnostic d'une feuille

3.4 Base de données

La base PostgreSQL contient trois tables, créées et versionnées par Flyway. Supprimer un compte supprime ses parcelles et ses prédictions ; supprimer une parcelle conserve ses prédictions dans l'historique.

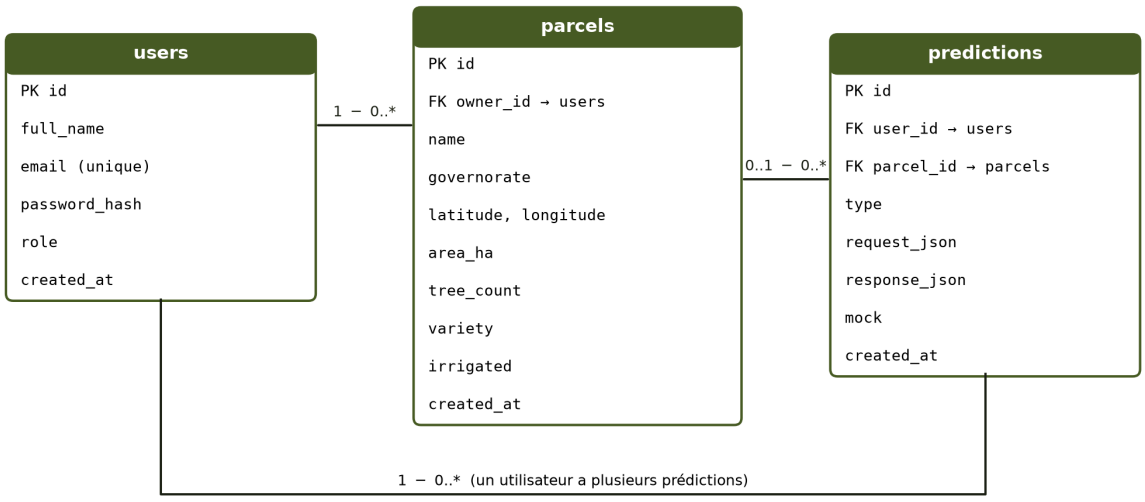


Figure 4 : Modèle de la base de données

3.5 API du backend

Le backend expose une API REST. Toutes les routes demandent un jeton JWT, sauf l'inscription, la connexion et la page d'état.

Route	Méthode	Rôle

/api/auth/register, /api/auth/login	POST	Inscription (agriculteur) et connexion
/api/users/me	GET	Profil de l'utilisateur connecté
/api/parcels, /api/parcels/{id}	GET, POST, PUT, DELETE	Parcelles de l'agriculteur
/api/ai/disease	POST (photo)	M1 · diagnostic d'une feuille
/api/parcels/{id}/irrigation	GET	M2 · irrigation et alertes
/api/parcels/{id}/yield	GET	M3 · récolte de la parcelle
/api/ai/price	GET	M4 · prix et conseil de vente
/api/ai/chat	POST	M5 · question à l'assistant
/api/parcels/{id}/count-trees	POST (image)	M6 · comptage des oliviers
/api/parcels/{id}/harvest-plan	POST	Plan de récolte M6 → M3 → M4
/api/history	GET	Historique des prédictions
/api/admin/stats, users, parcels, predictions	GET, DELETE	Back-office (rôle ADMIN)
/api/status	GET	État du backend, de la base et des modules

Tableau 6 : Principales routes de l'API du backend

3.6 Choix technologiques

Couche	Technologies	Justification
Frontend	Angular 21, TypeScript, SCSS	Framework structuré, adapté à une application à plusieurs espaces (agriculteur, administrateur)
Backend	Spring Boot 4.1, Java 21, Spring Security, JWT	Sécurité robuste, accès aux données simple (JPA), standard en entreprise
Base de données	PostgreSQL 17, Flyway	Base relationnelle fiable, schéma versionné
Services d'IA	Python, FastAPI	Écosystème de l'apprentissage automatique, API rapide et typée
Apprentissage	PyTorch, scikit-learn, statsmodels, Prophet, OpenCV, NumPy, pandas	Bibliothèques de référence pour chaque type de modèle
Langage naturel	Gemini, Groq en secours	Rédaction des réponses en derja, avec une solution de repli
Déploiement	Docker, Docker Compose	Neuf conteneurs lancés en une commande, sur n'importe quel PC

Tableau 7 : Choix technologiques

3.7 Conclusion

La conception repose sur un backend central, sécurisé, qui orchestre six microservices d'IA indépendants. Le chapitre suivant décrit comment chacun de ces modules a été réalisé.

Chapitre 4 : Réalisation des modules d'intelligence artificielle

4.1 Introduction

Ce chapitre présente les six modules d'IA avec la même structure, afin que le travail de chaque membre soit présenté de façon équitable : objectif, données, démarche et modèles comparés, résultats, intégration et limites. Il commence par les données utilisées et se termine par un tableau de synthèse.

4.2 Données utilisées

Toutes les données sont réelles et publiques. Les données brutes restent dans `data/raw/` sans jamais être modifiées, et les modèles entraînés dans `models/`, hors du dépôt Git.

Module	Source	Contenu	Période
M1	Kaggle, Olive Leaf Image Dataset (Uğuz et Uysal)	3 400 photos : feuille saine, œil de paon, Aculus olearius	—
M2	Open-Meteo : réanalyse ERA5 et prévisions	ET_0, pluie, températures de 12 gouvernorats oléicoles	2014 → 2026
M3	FAOSTAT et ERA5	Production d'olives de la Tunisie, météo saisonnière	1961 → 2024
M4	FMI (série FRED POLVOILUSDM) et Banque mondiale	Prix mensuel de l'huile extra vierge, taux de change TND/USD	1992 → 2026
M5	Questions écrites par l'équipe	500 questions annotées, 9 intentions, en arabizi, arabe et français	—
M6	Méthode classique, sans entraînement	Images vues du ciel fournies par l'utilisateur ; vergers de test générés	—

Tableau 8 : Données utilisées par les six modules

4.3 M1 · Diagnostic des maladies des feuilles

> **Responsable :** Omar Azouz · **Type d'IA :** vision par ordinateur, classification d'images · **Modèle servi :** MobileNetV3-Large

4.3.1 Objectif

Permettre à l'agriculteur de photographier une feuille et de savoir si elle est saine ou atteinte d'œil de paon ou d'Aculus olearius, avec un conseil adapté.

4.3.2 Données

Les 3 400 photos sont découpées en 2 380 pour l'entraînement, 512 pour la validation et 508 pour le test. Le découpage est groupé par feuille d'origine : aucune variante d'une même feuille ne peut passer de l'entraînement au test, ce qui donnerait des résultats trop optimistes.

4.3.3 Démarche et modèles comparés

- **Référence** : un petit réseau convolutif de trois blocs, entraîné de zéro.
- **Modèle** : MobileNetV3-Large préentraîné sur ImageNet, affiné en deux phases (la tête seule, puis le réseau complet), avec des augmentations qui imitent une photo prise au champ et une pondération des classes.
- **Fiabilité** : calibration par température (T = 0,49), qui fait passer l'erreur de calibration de 0,045 à 0,020. Sous 70 % de confiance, aucun diagnostic n'est donné et l'application demande une autre photo.

4.3.4 Résultats

Modèle (508 photos de test)	Précision	F1 macro
Petit CNN (référence)	75,2 %	0,741
MobileNetV3	**97,1 %**	**0,969**
MobileNetV3, photos acceptées (seuil 70 %)	98,2 %	—

Tableau 9 : Résultats du module M1

Le F1 par classe est de 0,964 pour les feuilles saines, 0,977 pour l'œil de paon et 0,967 pour Aculus olearius. Seules 15 photos sur 508 sont refusées par le seuil de confiance. La carte Grad-CAM montre que le modèle regarde bien les taches de la feuille, et non le fond de la photo.

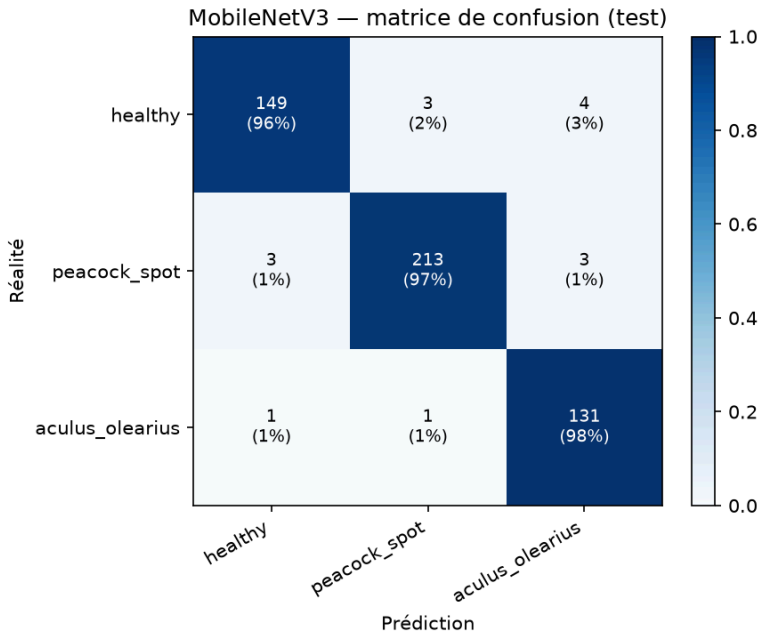


Figure 5 : Matrice de confusion de MobileNetV3 sur les photos de test

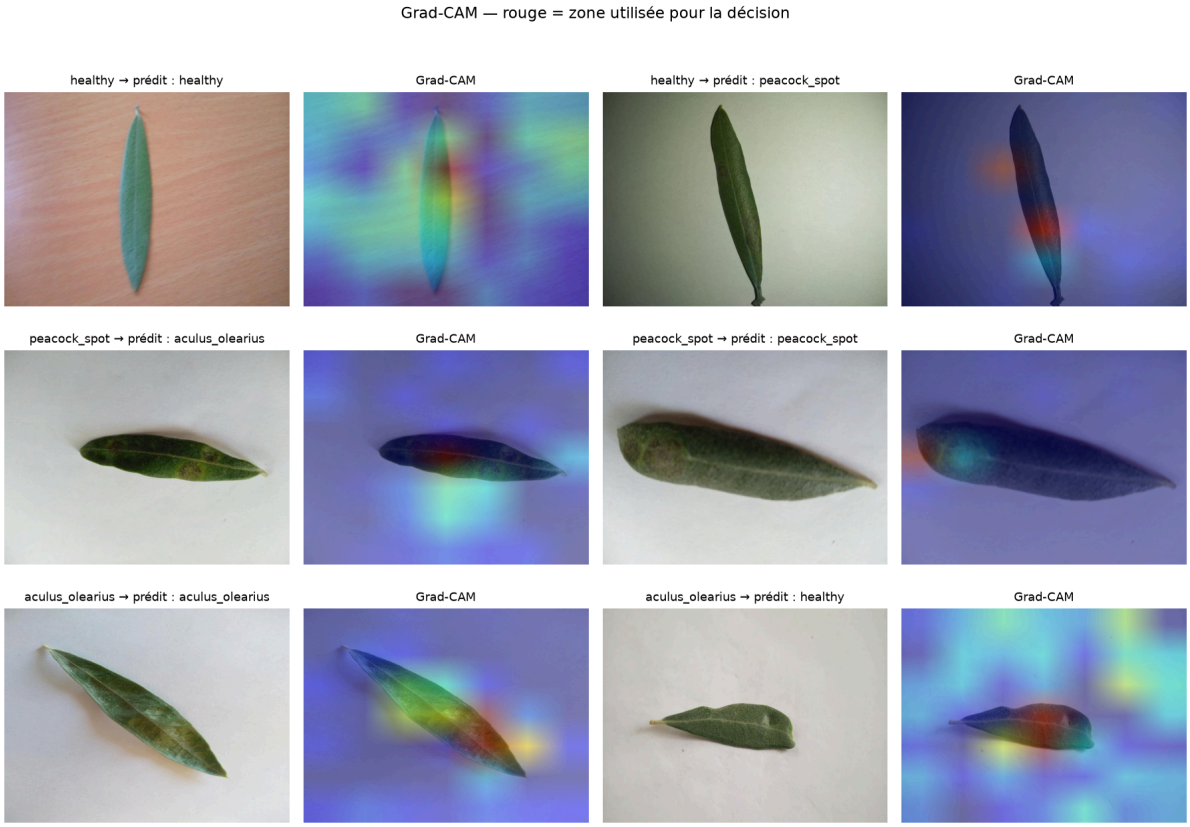


Figure 6 : Cartes Grad-CAM : en rouge, la zone utilisée pour décider

4.3.5 Intégration

Le modèle est exporté en TorchScript et chargé par un service FastAPI, avec la même version de PyTorch (2.11) qu'à l'entraînement. L'écran « Diagnostic » affiche la maladie, la confiance, la carte Grad-CAM et le conseil, ou le message « Photo peu claire » sous le seuil.

4.3.6 Limites

Le jeu de données ne couvre que trois classes et des photos prises sur fond uni ; d'autres maladies de l'olivier et des photos prises directement sur l'arbre enrichiraient le modèle.

4.4 M2 · Irrigation et alertes météo

> **Responsable :** Khadija Ben Ayed · **Type d'IA :** séries temporelles · **Modèle servi :** LSTM

4.4.1 Objectif

Dire chaque jour à l'agriculteur combien arroser sa parcelle et le prévenir d'un risque de gel ou de canicule.

4.4.2 Données

Météo journalière ERA5 de 12 gouvernorats oléicoles, de 2014 à 2026. Le découpage est chronologique : entraînement de 2014 à 2021, validation en 2022, test de 2023 à 2026, soit 16 356 prévisions évaluées. La variable prévue est l'évapotranspiration de référence (ET_0), c'est-à-dire l'eau perdue par le sol et les plantes.

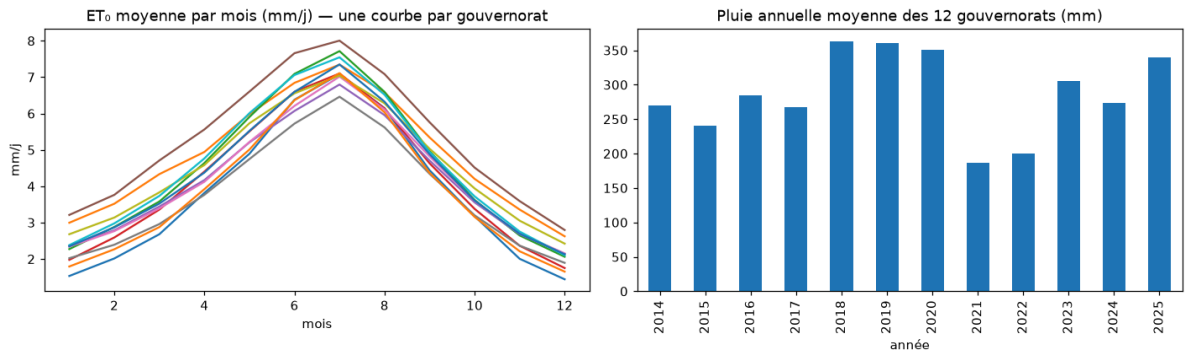


Figure 7 : Exploration des données : ET_0 mensuelle par gouvernorat et pluie annuelle

4.4.3 Démarche et modèles comparés

- **Références** : « même jour l'an dernier » et persistance sur 7 jours.
- **Modèles** : Prophet et un LSTM global qui reçoit 30 jours de météo et prévoit l'ET_0 des 16 jours suivants.
- **Besoin en eau** : formule FAO-56, besoin = $ET_0 \times 0{,}65 - 0{,}8 \times$ pluie ; irrigation conseillée au-dessus de 2 mm, convertie en litres par arbre selon la surface et le nombre d'arbres.
- **Alertes** : gel si la température minimale prévue est inférieure ou égale à 0 °C, canicule si la maximale atteint 40 °C.

4.4.4 Résultats

Modèle (test 2023 → 2026, horizons 1 à 7 jours)	MAE de l'ET_0
Même jour l'an dernier (référence)	1,06 mm/j
Persistance 7 jours	0,80 mm/j
LSTM	**0,71 mm/j**
Prophet / LSTM (sur les dates communes)	0,666 / 0,657 mm/j

Tableau 10 : Résultats du module M2

Le LSTM réduit l'erreur de 33 % par rapport à la référence. Il fait légèrement mieux que Prophet tout en étant bien plus rapide à exécuter. Contrôle réel : le 17 octobre 2026 à Sfax, le LSTM prévoyait une ET_0 de 3,3 à 3,7 mm/j, contre 3,0 à 3,4 mm/j pour la prévision numérique d'Open-Meteo.

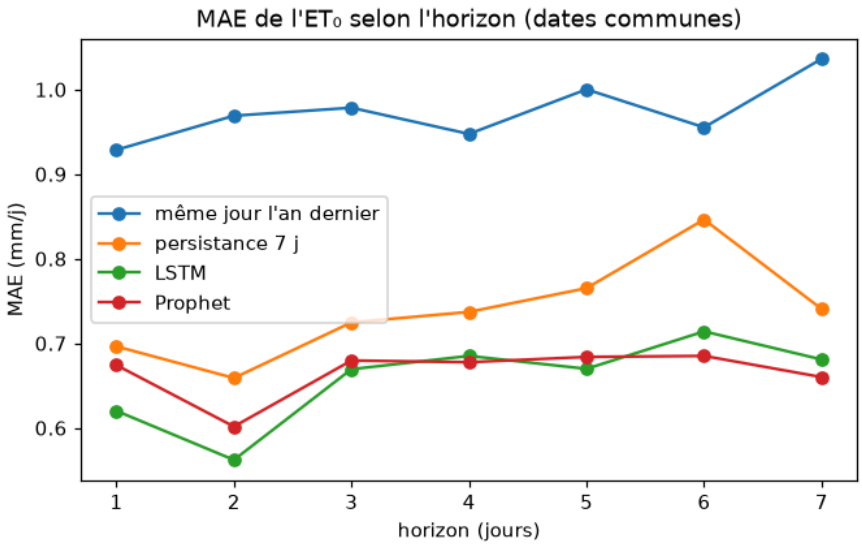


Figure 8 : Erreur de prévision de l'ET_0 selon l'horizon

4.4.5 Intégration

Le LSTM est exécuté en NumPy dans le service FastAPI (écart avec PyTorch de $1{,}6 \times 10^{-6}$ mm/j), ce qui évite d'installer PyTorch. Le service récupère les 30 derniers jours et les prévisions auprès d'Open-Meteo pour les coordonnées de la parcelle. La page « Irrigation » affiche le besoin jour par jour et les alertes.

4.4.6 Limites

Le coefficient cultural (0,65) est le même pour toutes les parcelles ; l'adapter à l'âge des arbres et au type de sol rendrait le conseil plus précis.

4.5 M3 · Prévision de la récolte

> **Responsable :** Med Amine Bejaoui · **Type d'IA :** régression sur données tabulaires · **Modèle servi :** Random Forest

4.5.1 Objectif

Estimer avant la saison la production d'olives attendue, pour le pays et pour la parcelle de l'agriculteur.

4.5.2 Données

Production, surface et rendement de la Tunisie publiés par FAOSTAT de 1961 à 2024, croisés avec la météo saisonnière ERA5. Les productions par gouvernorat (rapports ONAGRI en PDF) ne sont pas disponibles en données ouvertes : conformément au plan B prévu par le plan de projet, le modèle prévoit la production nationale.

4.5.3 Démarche et modèles comparés

- **Variables** : production des deux saisons précédentes (alternance des bonnes et des mauvaises années), tendance, pluie de septembre à mai, pluie d'été, jours à 35 °C ou plus en avril-mai (floraison), température maximale moyenne de l'été.

- **Validation** chronologique glissante de 2005 à 2024 : chaque saison est prévue par un modèle entraîné sur les seules saisons précédentes.
- **Modèles comparés** : « comme la saison précédente », régression linéaire (référence du plan), Ridge, Gradient Boosting et Random Forest.

4.5.4 Résultats

Modèle (validation glissante 2005 → 2024)	MAPE
Comme la saison précédente	60,7 %
Régression linéaire (référence)	55,7 %
Ridge	52,3 %
Gradient Boosting	46,0 %
Random Forest	**40,6 %**

Tableau 11 : Résultats du module M3

Le Random Forest réduit l'erreur de 27 % par rapport à la régression linéaire. L'erreur reste élevée parce que la production tunisienne varie énormément d'une année à l'autre, de 0,7 million de tonnes (2021) à 2,0 millions (2020). La tendance et la pluie de septembre à mai sont les variables les plus utiles.

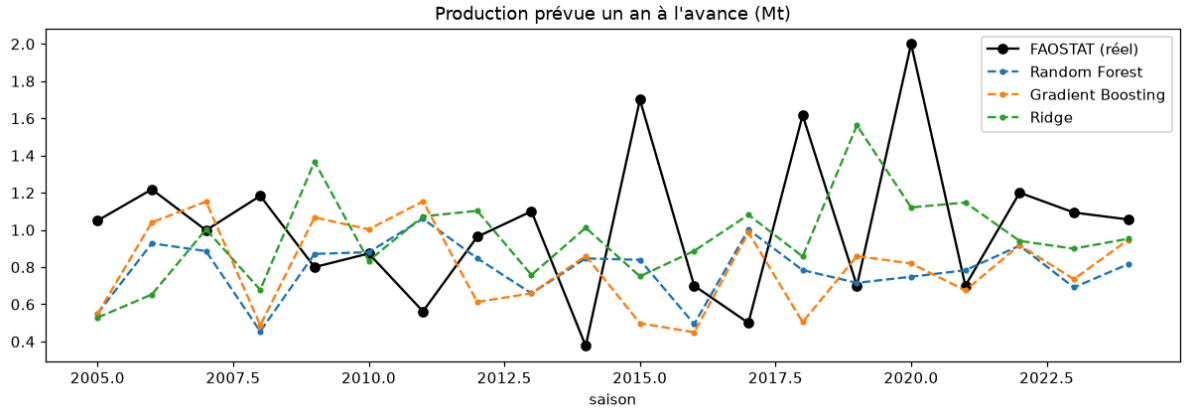


Figure 9 : Production prévue un an à l'avance, comparée à la production réelle

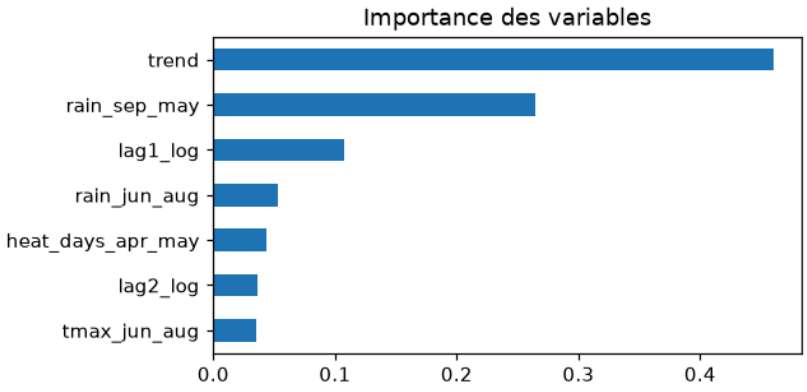


Figure 10 : Importance des variables du Random Forest

4.5.5 Intégration

Pour la parcelle, le service multiplie le rendement national par hectare par la surface de la parcelle et donne une fourchette tirée des vraies erreurs du test (−53 % à +90 %). Pour une saison déjà publiée par la FAO, il renvoie le chiffre réel. L'estimation alimente aussi le plan de récolte.

4.5.6 Limites

Le modèle est national : extraire les rapports ONAGRI par gouvernorat permettrait d'entraîner le même pipeline par région.

4.6 M4 · Prix de l'huile et moment de vente

> **Responsable :** Emna Dorai · **Type d'IA :** séries temporelles · **Modèle servi :** SARIMA $(0,1,1)(1,0,1)_{12}$

4.6.1 Objectif

Prévoir le prix de l'huile d'olive sur les prochains mois et conseiller à l'agriculteur de vendre maintenant ou de stocker.

4.6.2 Données

Prix mondial mensuel de l'huile d'olive extra vierge (FMI), converti en dinars par kilogramme avec le taux de change annuel de la Banque mondiale, de 1992 à juillet 2026. Le découpage est chronologique : entraînement jusqu'en 2016, validation de 2017 à 2020, test de 2021 à 2026, période qui inclut la flambée des prix de 2023-2024 puis leur chute.

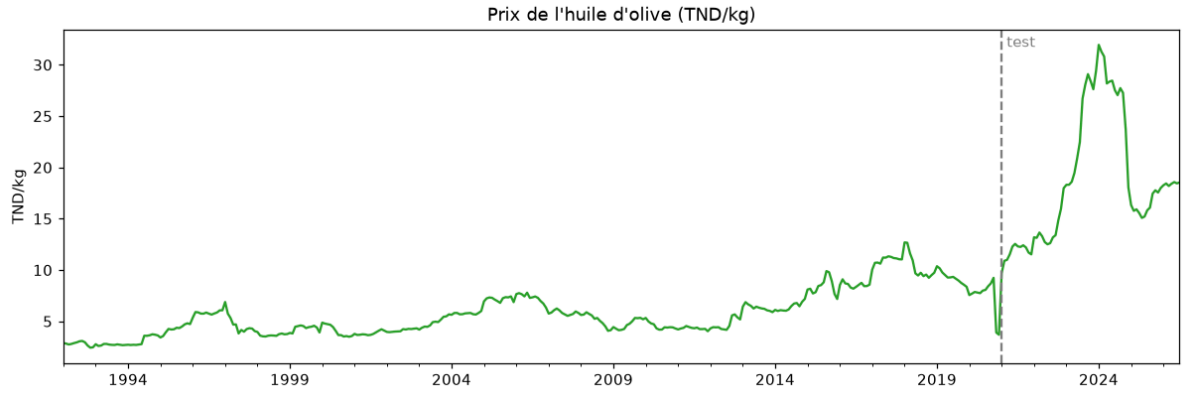


Figure 11 : Prix de l'huile d'olive en TND/kg depuis 1992

4.6.3 Démarche et modèles comparés

- **Modèles comparés** : prévision naïve (le prix ne change pas), LSTM et SARIMA, sur des horizons de 1 à 6 mois.
- **Conseil** : stocker seulement si le prix prévu, diminué du coût de stockage (0,15 TND/kg par mois), dépasse le prix du jour de plus d'une marge de prudence.
- **Marge de prudence** : 4 %, choisie sur la période de validation, jamais sur le test.

4.6.4 Résultats

Évaluation sur le test 2021 → 2026	Résultat

MAPE naïf (référence)	11,73 %
MAPE LSTM	11,77 %
MAPE SARIMA	**11,33 %**
Gain moyen du conseil sans marge (61 décisions)	−0,47 TND/kg
Gain moyen du conseil avec la marge de 4 %	**+0,10 TND/kg**

Tableau 12 : Résultats du module M4

SARIMA est le seul modèle à battre la prévision naïve ; le LSTM ne fait pas mieux qu'elle. L'intervalle à 80 % contient la vraie valeur dans 80 à 91 % des cas. Surtout, la marge de prudence transforme un conseil perdant en conseil gagnant : 9 conseils « stocker » sur 10 sont rentables.

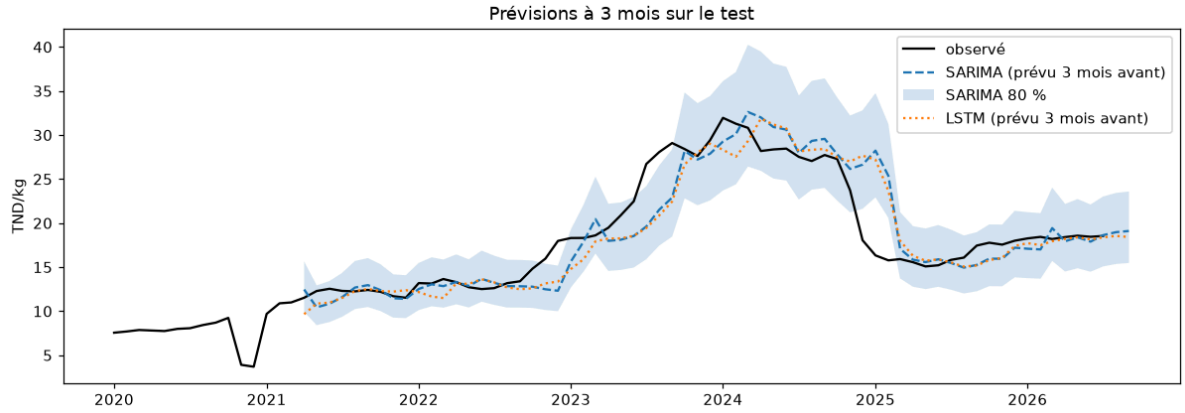


Figure 12 : Prévisions à 3 mois sur la période de test, avec l'intervalle à 80 %

4.6.5 Intégration

Le service réajuste SARIMA au démarrage sur toute la série disponible. La page « Prix » affiche la courbe de prévision, l'intervalle de confiance et le conseil de vente, qui sert aussi de dernière étape au plan de récolte.

4.6.6 Limites

Le prix utilisé est le prix mondial de référence, plus élevé que le prix payé au producteur tunisien ; le pipeline peut utiliser les prix du Conseil oléicole international dès qu'ils sont saisis.

4.7 M5 · Assistant conversationnel en derja

> **Responsable :** Rima Jenhani · **Type d'IA :** traitement du langage, classification et LLM · **Modèle servi :** TF-IDF caractères et régression logistique, puis Gemini

4.7.1 Objectif

Permettre à l'agriculteur de poser une question écrite librement, en derja, en arabe ou en français, et recevoir une réponse claire en derja, fondée sur les vrais chiffres des autres modules.

4.7.2 Données

Nous avons écrit et annoté 500 questions réparties en 9 intentions : maladie, irrigation, météo, récolte, prix, comptage, conseil général, salutation et hors sujet. Les questions mêlent arabizi

(derja en lettres latines, par exemple « 9adech »), arabe et français. Un test indépendant de 25 questions, sans aucun recouvrement avec l'entraînement, sert à vérifier le modèle final.

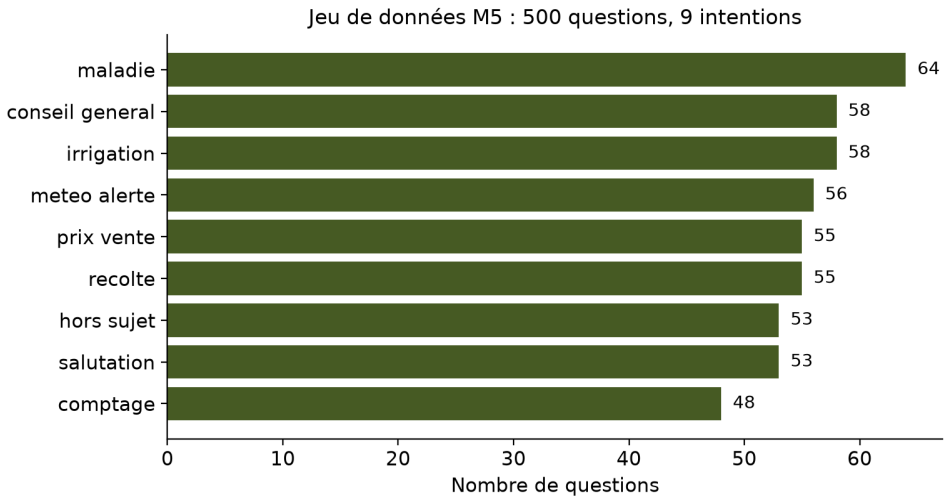


Figure 13 : Répartition des 500 questions par intention

4.7.3 Démarche et modèles comparés

- **Référence** : TF-IDF sur les mots et régression logistique.
- **Modèle** : TF-IDF sur des groupes de caractères, qui absorbe les orthographes variables de l'arabizi (zitoun, zitouni).
- **Clarification** : sous 0,35 de confiance, l'assistant pose une question au lieu de deviner ; il ne répond jamais « hors sujet » à une question qui parle d'olives.
- **Réponse** : le backend appelle le module concerné (M2, M3, M4…), puis Gemini, avec Groq en secours, rédige la réponse en derja à partir de ces chiffres, sans en inventer.

4.7.4 Résultats

Modèle	F1 macro (validation croisée 5 plis)	Test indépendant
TF-IDF mots (référence)	0,72	16 / 25
TF-IDF caractères	**0,75**	**19 / 25**
TF-IDF mots + caractères	0,75	18 / 25

Tableau 13 : Résultats du module M5

Le modèle à caractères gagne trois points de F1 et trois bonnes réponses au test indépendant. Avec la règle de clarification, environ trois questions sur quatre reçoivent une réponse directe, avec au moins 85 % de justesse.

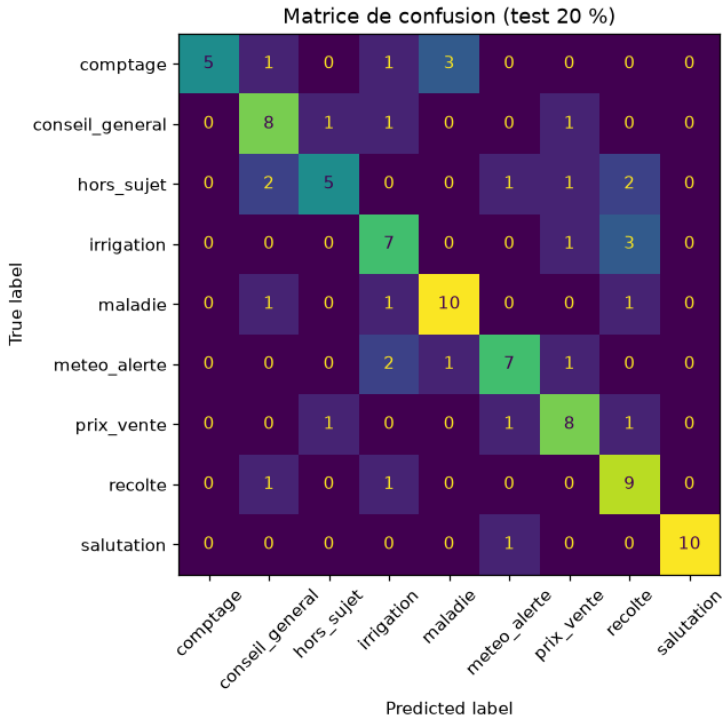


Figure 14 : Matrice de confusion des intentions sur 20 % de questions mises de côté

4.7.5 Intégration

La page « Assistant » propose une conversation. Le modèle de langage reçoit la question et les chiffres des modules, jamais le nom ni l'e-mail de l'agriculteur. Sans modèle de langage disponible, des phrases modèles prennent le relais.

4.7.6 Limites

Le jeu de 500 questions reste modeste ; l'enrichir et comparer un modèle de type BERT (SetFit), qui demande un processeur graphique, sont les prochaines étapes.

4.8 M6 · Comptage des oliviers

> **Responsable :** Aziz Dlissi · **Type d'IA :** vision par ordinateur, détection d'objets · **Modèle servi :** détecteur OpenCV

4.8.1 Objectif

Compter automatiquement les oliviers d'une parcelle à partir d'une image vue du ciel, pour alimenter le plan de récolte.

4.8.2 Données

La méthode retenue est une méthode classique de vision, la référence prévue par le plan de projet : elle ne demande pas de données d'entraînement. Pour l'évaluer avec un nombre d'arbres connu, nous générons des vergers de test réalistes (sol texturé, couronnes de tailles variées, couronnes qui se touchent).

4.8.3 Démarche

1. Calcul d'un indice de végétation par pixel : vert en excès (2G − R − B) et assombrissement, car les couronnes sont plus sombres que le sol.
2. Seuil automatique d'Otsu, puis ouverture et fermeture morphologiques pour nettoyer le masque.
3. Estimation du rayon typique d'une couronne (rayon médian des taches isolées).
4. Transformée de distance et recherche des maxima locaux espacés d'au moins un rayon : un arbre par maximum, ce qui sépare les couronnes qui se touchent.

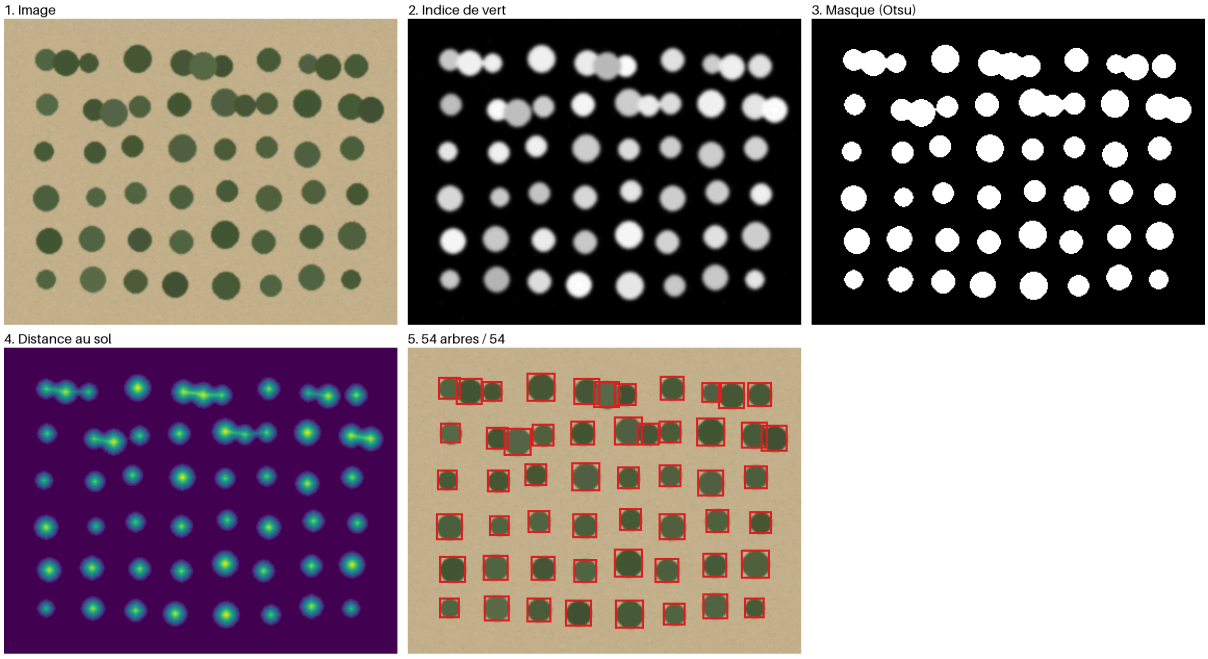


Figure 15 : Les étapes du détecteur sur un verger de test aux couronnes collées

4.8.4 Résultats

Verger de test	Arbres réels	Arbres détectés
4 × 4	16	16
6 × 6	36	36
8 × 6	48	48
10 × 8	80	80
6 × 6 + 6 couronnes collées	42	42
8 × 8 + 10 couronnes collées	74	74

Tableau 14 : Résultats du module M6

Le détecteur trouve le nombre exact d'arbres sur les six vergers de test, y compris quand les couronnes se touchent. Les tests automatiques exigent une erreur inférieure ou égale à 5 % pour des arbres séparés, à 10 % pour des couronnes collées, et zéro arbre sur un sol nu. Le service en ligne a trouvé 52 arbres sur un verger test qui en compte 52.

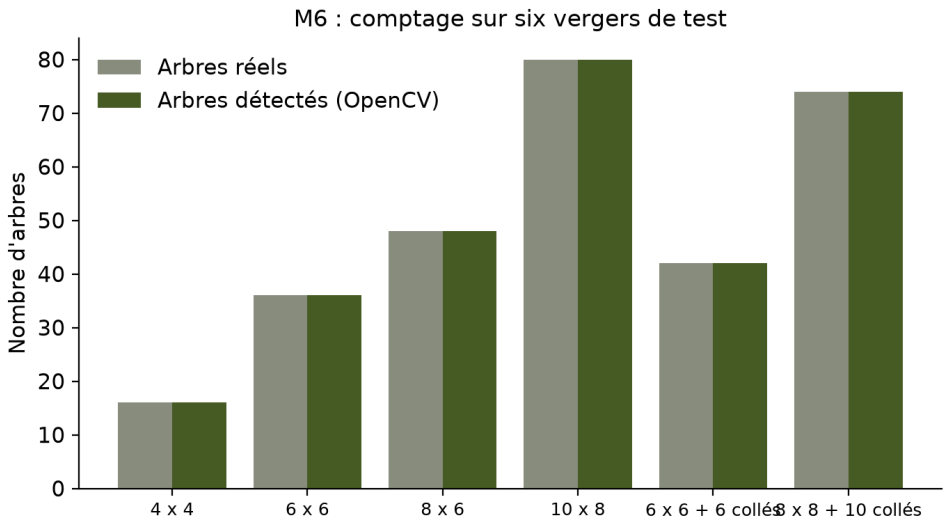


Figure 16 : Arbres réels et arbres détectés sur six vergers de test

4.8.5 Intégration

Le service renvoie le nombre d'arbres, la position de chaque couronne et l'image annotée. Le résultat alimente le plan de récolte : arbres (M6), puis kilos d'olives (M3), puis conseil de vente (M4).

4.8.6 Limites

La méthode est évaluée sur des images générées. La suite prévue est d'entraîner YOLOv8 sur le jeu OliveTreeCrownsDb et de le comparer à cette méthode (mAP@0.5) ; le service l'utilisera automatiquement dès que ses poids existeront.

4.9 Synthèse des résultats

Module	Métrique	Référence simple	Modèle servi	Gain
M1 Maladies	Précision	Petit CNN : 75,2 %	MobileNetV3 : 97,1 %	+21,9 points
M2 Irrigation	MAE de l'ET_0	An dernier : 1,06 mm/j	LSTM : 0,71 mm/j	−33 % d'erreur
M3 Récolte	MAPE	Régression linéaire : 55,7 %	Random Forest : 40,6 %	−27 % d'erreur
M4 Prix	MAPE / gain	Naïf : 11,7 %	SARIMA : 11,3 %	+0,10 TND/kg
M5 Assistant	F1 macro	TF-IDF mots : 0,72	TF-IDF caractères : 0,75	19/25 au test
M6 Comptage	Arbres trouvés	—	OpenCV : 296 / 296	Erreur 0 % sur 6 vergers

Tableau 15 : Synthèse : chaque modèle servi comparé à sa référence

4.10 Conclusion

Les six modules reposent sur des données réelles et une évaluation honnête : chaque modèle servi bat sa référence simple sur des données qu'il n'a jamais vues. Le chapitre suivant montre comment ces modules sont réunis dans une même application.

Chapitre 5 : Réalisation de l'application, tests et déploiement

5.1 Introduction

Ce chapitre présente l'environnement de travail, l'application réalisée, la sécurité, les tests et le déploiement.

5.2 Environnement de travail

- **Matériel** : ordinateurs portables personnels sous Windows, sans carte graphique dédiée ; c'est pourquoi nous avons privilégié des modèles légers.
- **Logiciels** : Visual Studio Code et IntelliJ IDEA, Jupyter pour les notebooks, Docker Desktop, Git.
- **Entraînement** : images Docker dédiées contenant les bibliothèques d'apprentissage, pour que chaque notebook s'exécute de la même façon sur tous les postes.

5.3 Présentation de l'application

Un même écran de connexion mène l'agriculteur à son espace et l'administrateur au back-office. L'interface est organisée par rubriques dans une barre latérale, avec une palette olive et or et des formats français.

Espace	Page	Ce qu'on y fait
Agriculteur	Accueil	Résumé de l'exploitation, accès aux six outils, activité récente
Agriculteur	Parcelles	Ajout de parcelles ; pour chacune : irrigation (M2), récolte (M3), comptage (M6), plan de récolte
Agriculteur	Diagnostic	Photo d'une feuille et diagnostic (M1)
Agriculteur	Irrigation	Besoin en eau jour par jour et alertes (M2)
Agriculteur	Prix	Courbe de prévision et conseil de vente (M4)
Agriculteur	Assistant	Conversation en derja (M5)
Agriculteur	Historique	Toutes ses prédictions
Administrateur	Tableau de bord	Utilisateurs, parcelles, oliviers, activité sur 30 jours, état des six modèles
Administrateur	Utilisateurs, Parcelles, Prédictions	Recherche, suppression des comptes agriculteurs et des parcelles, détail des appels aux modèles

Tableau 16 : Pages de l'application

5.4 Captures de l'application

Les captures suivantes ont été prises le 18 octobre 2026 sur l'application lancée en local, avec un compte agriculteur réel puis le compte administrateur. Tous les résultats affichés proviennent des modèles entraînés, sans mode démo.

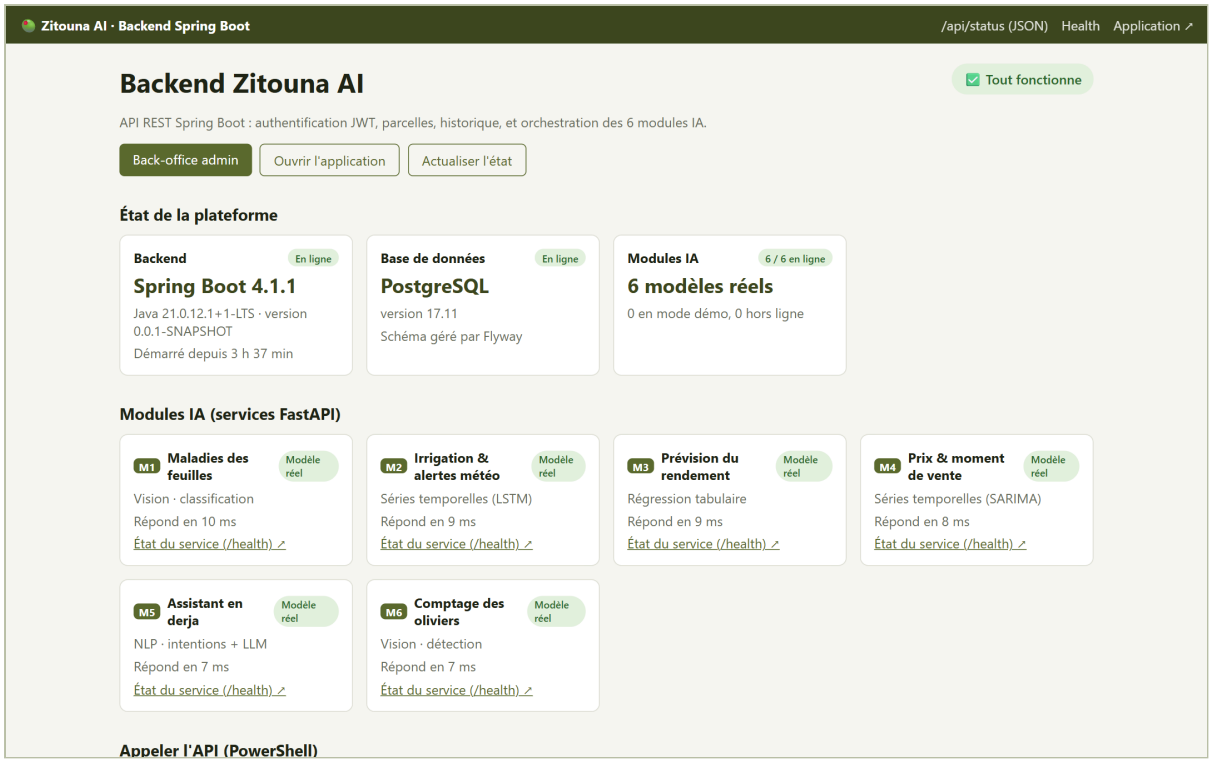


Figure 17 : Page d'état du backend Spring Boot : base de données et six modèles réels en ligne

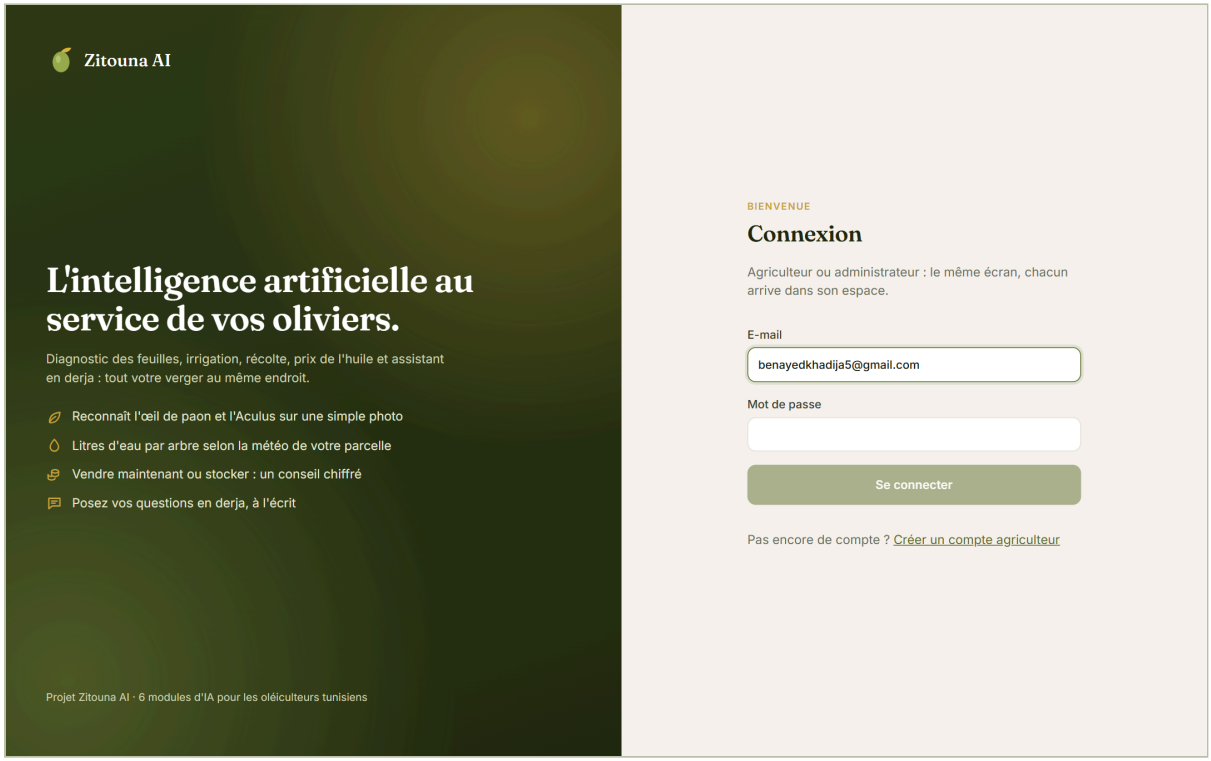


Figure 18 : Écran de connexion commun aux agriculteurs et à l'administrateur

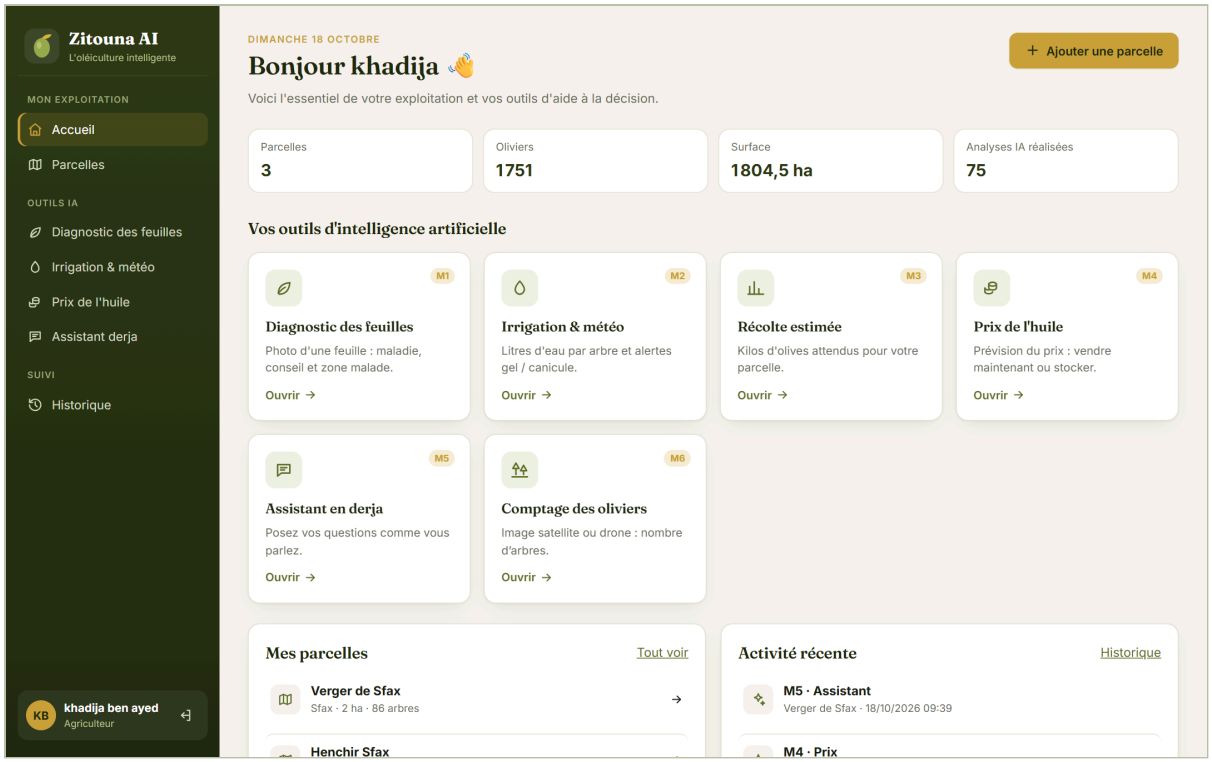


Figure 19 : Accueil de l'agriculteur : résumé de l'exploitation et accès aux six outils d'IA

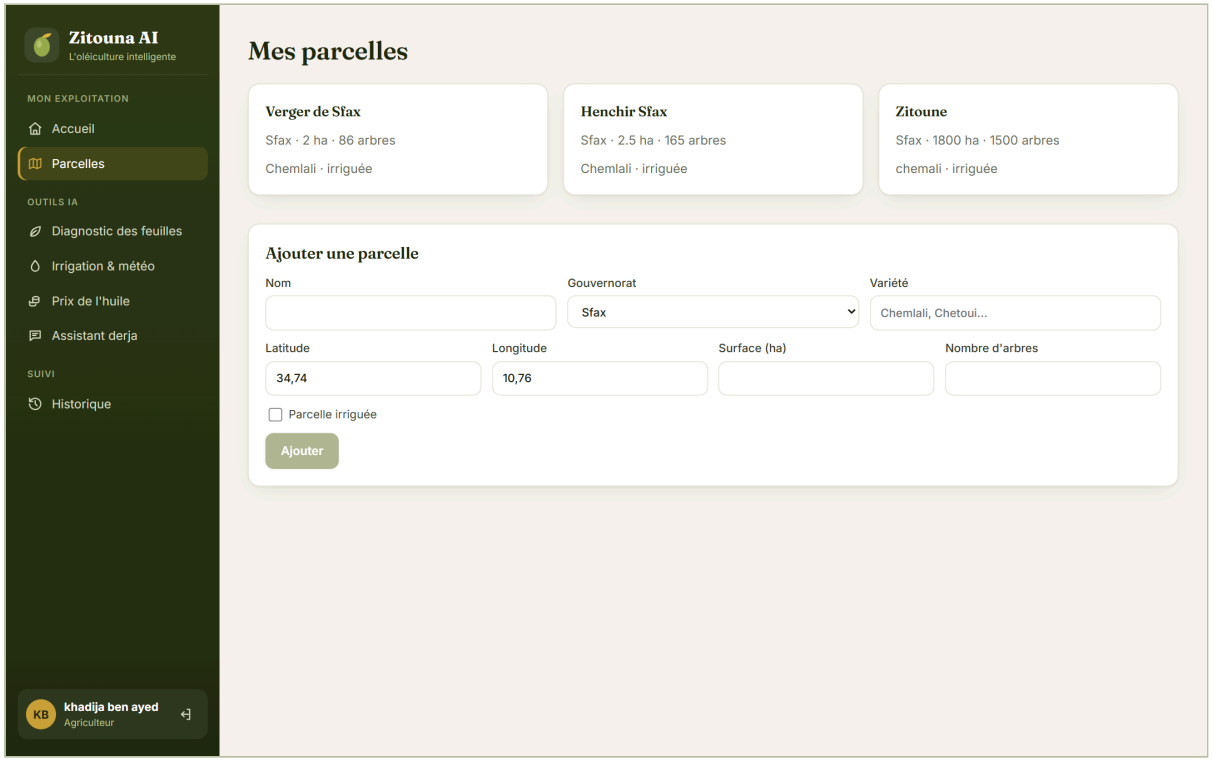


Figure 20 : Liste des parcelles et formulaire d'ajout

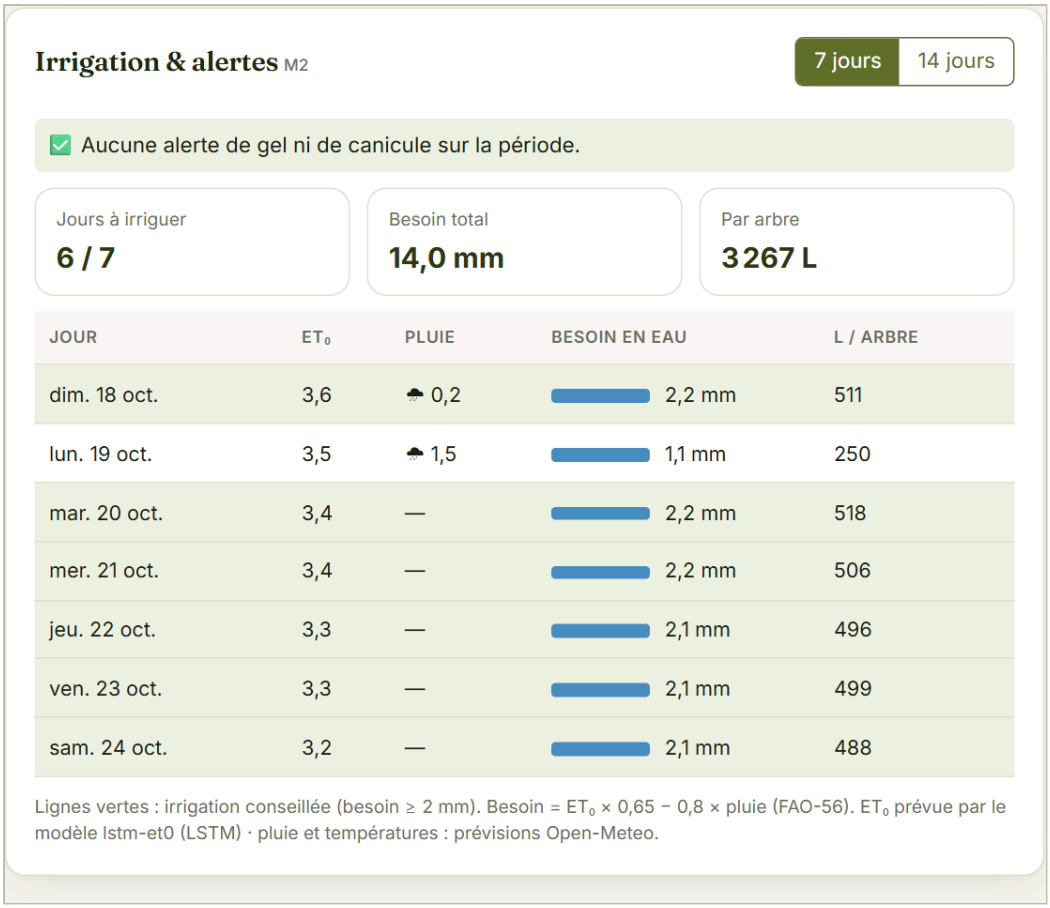


Figure 21 : M2 · Conseil d'irrigation sur 7 jours pour la parcelle « Verger de Sfax »

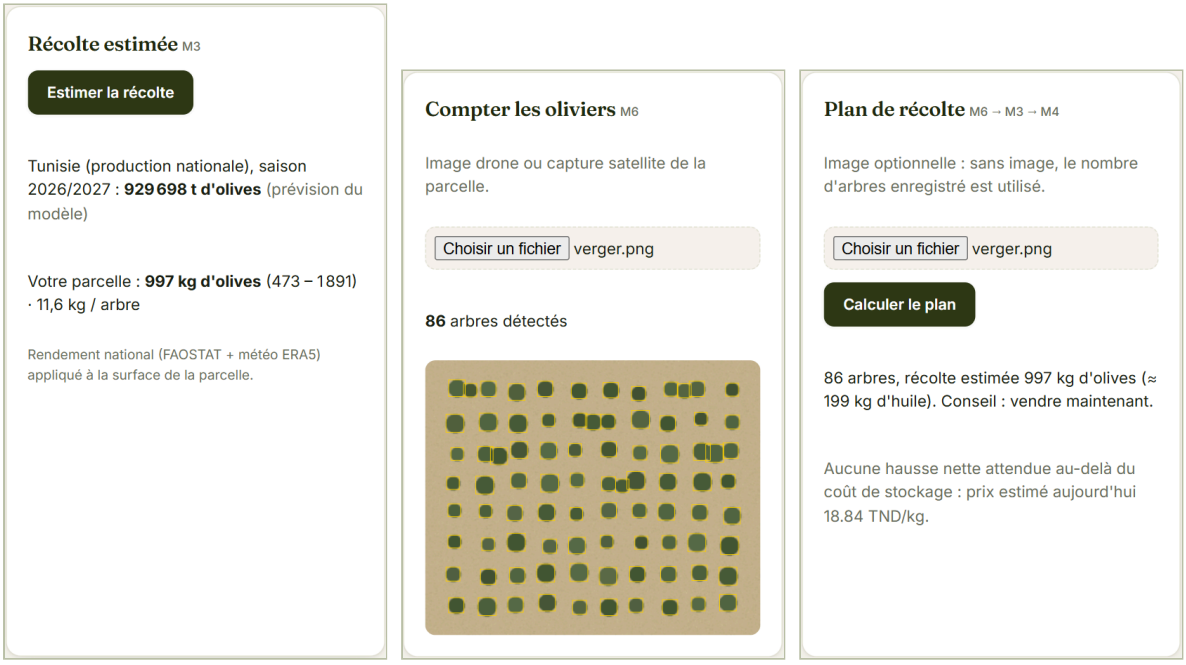


Figure 22 : M3 · Récolte estimée (997 kg), M6 · 86 arbres détectés et plan de récolte M6 → M3 → M4

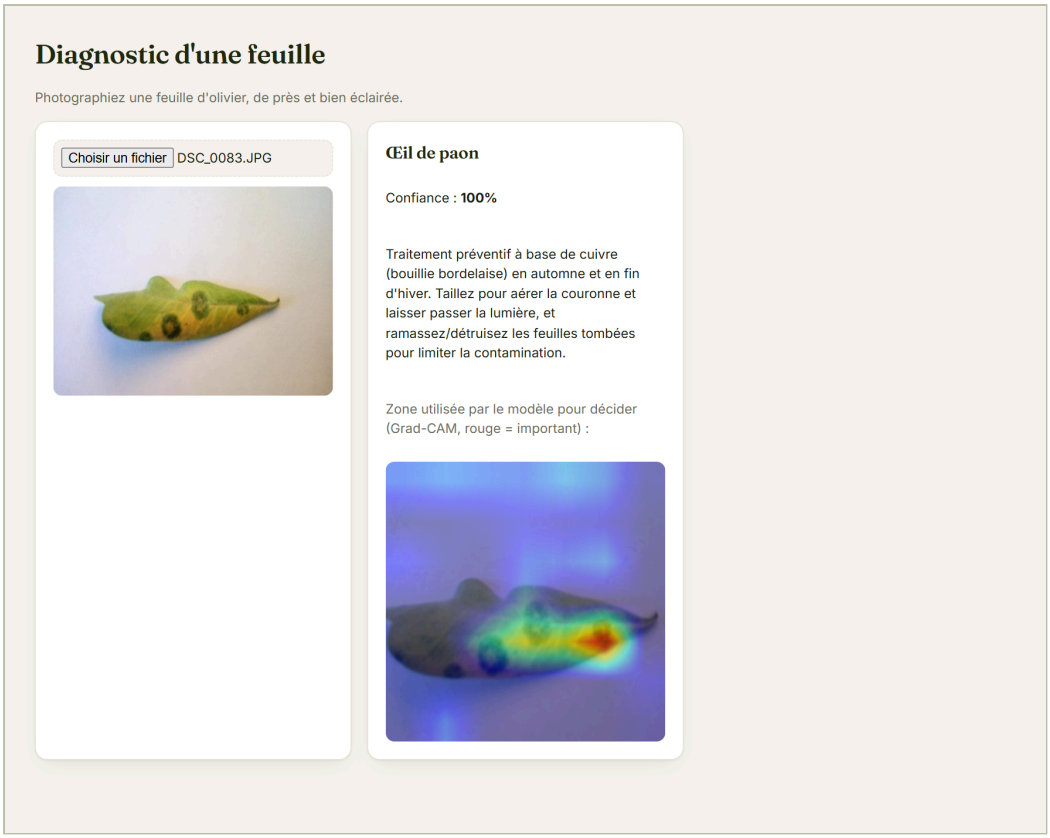


Figure 23 : M1 · Diagnostic d'une feuille : œil de paon détecté et carte Grad-CAM

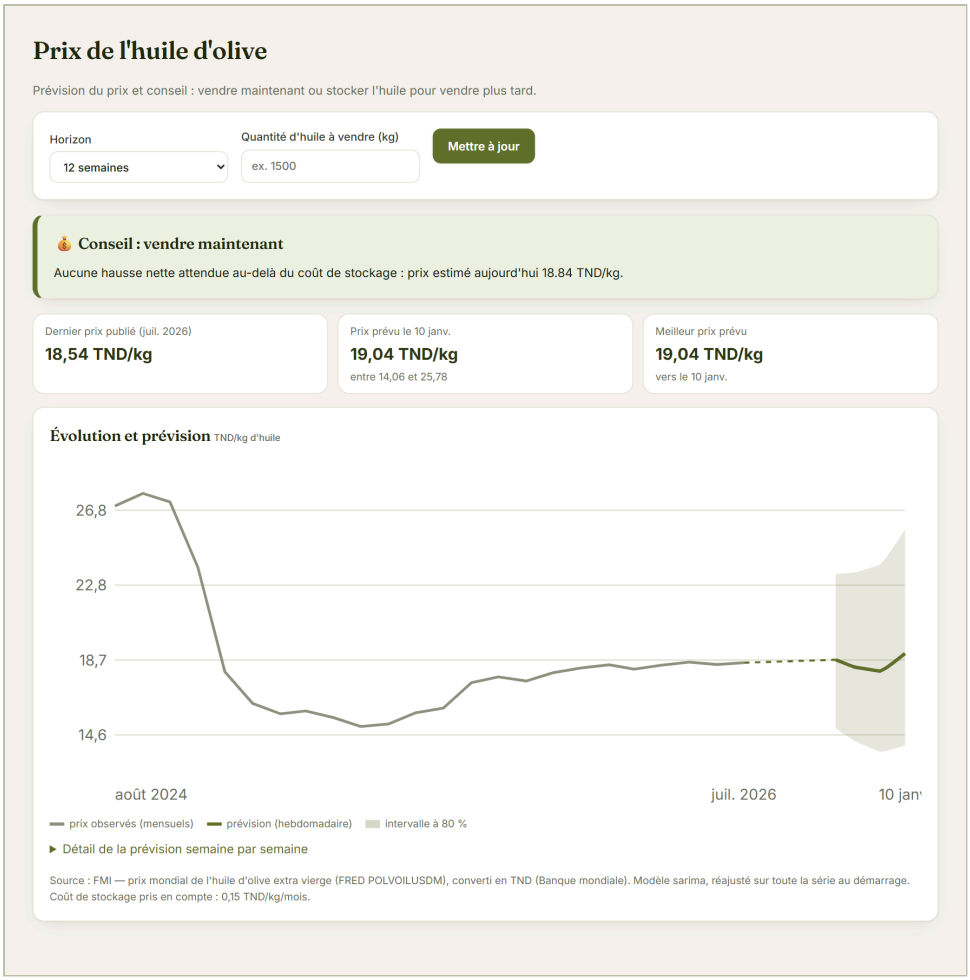


Figure 24 : M4 · Prévision du prix de l'huile et conseil de vente

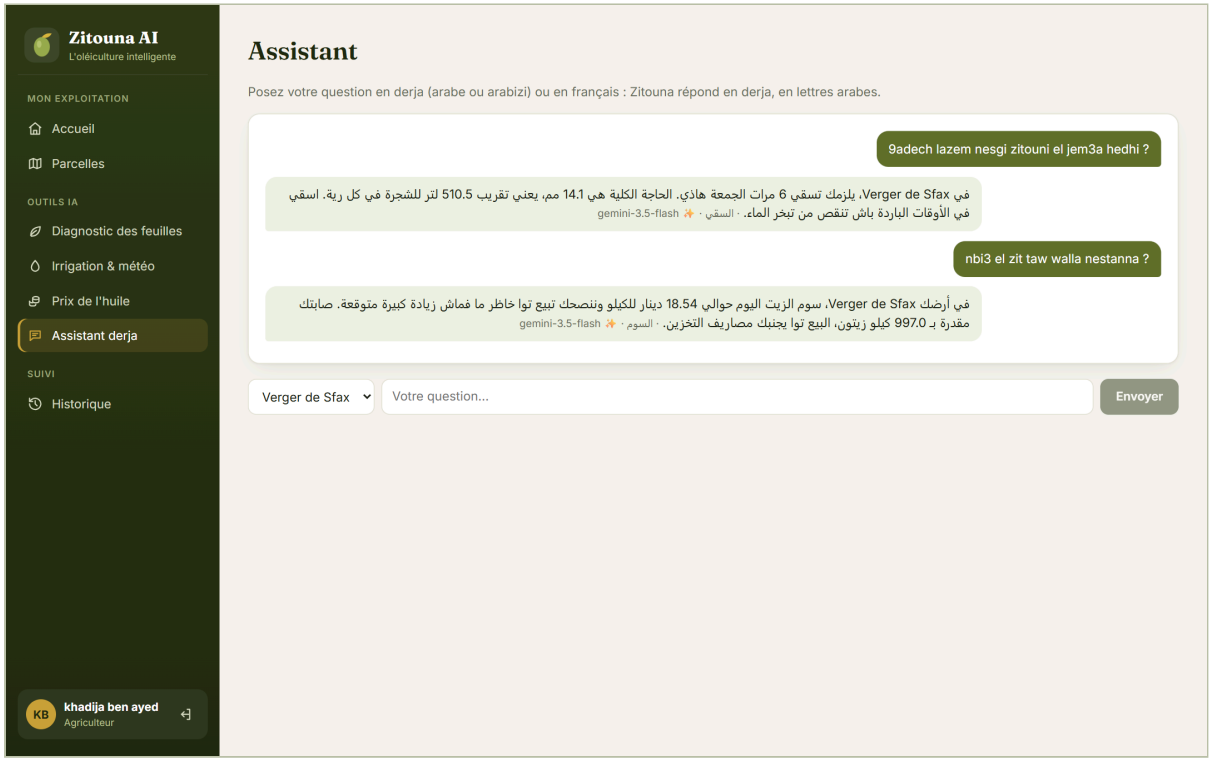


Figure 25 : M5 · Questions en derja et réponses de l'assistant rédigées par Gemini

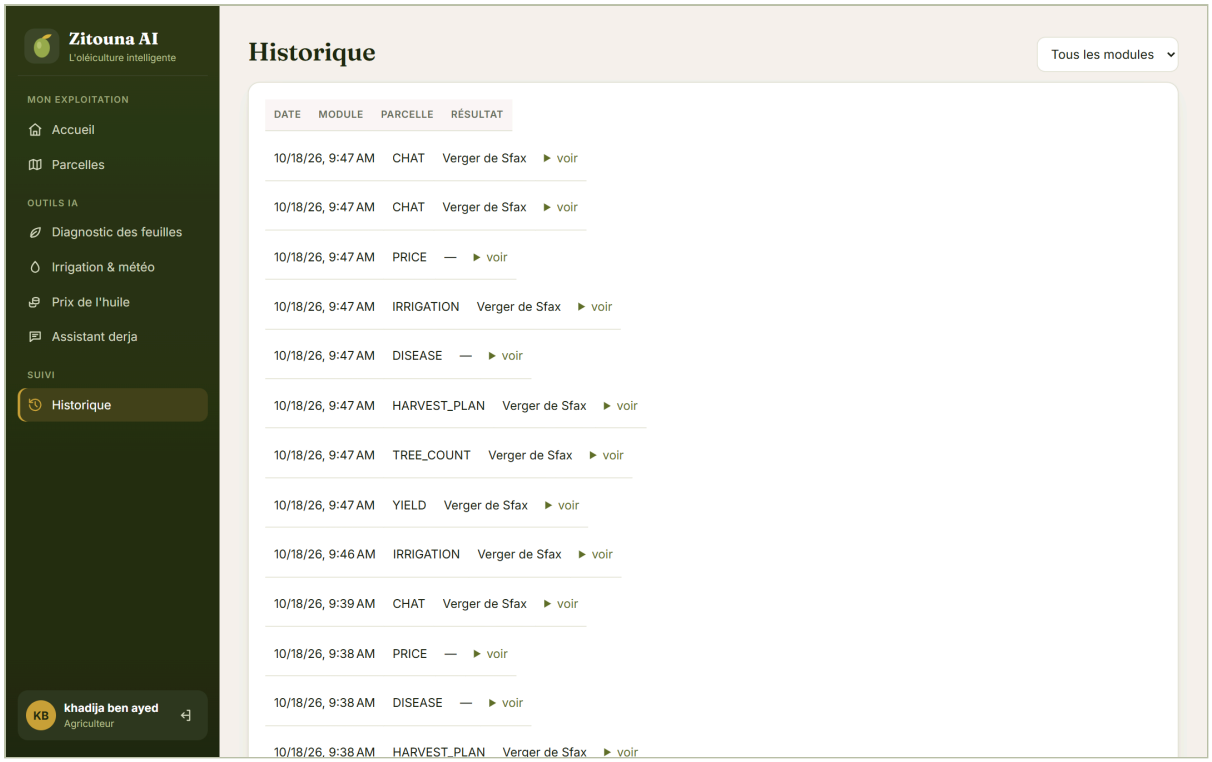


Figure 26 : Historique des prédictions de l'agriculteur

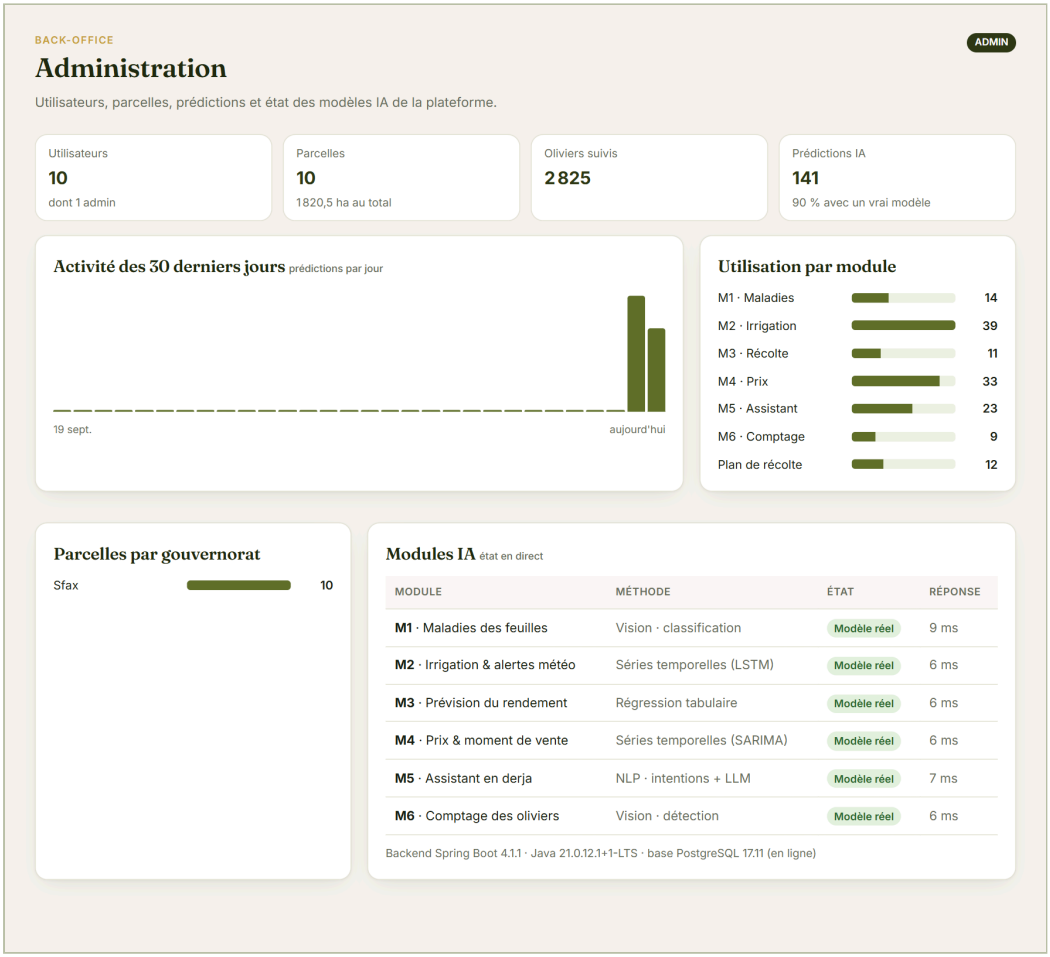


Figure 27 : Back-office : tableau de bord de l'administrateur et état des six modèles

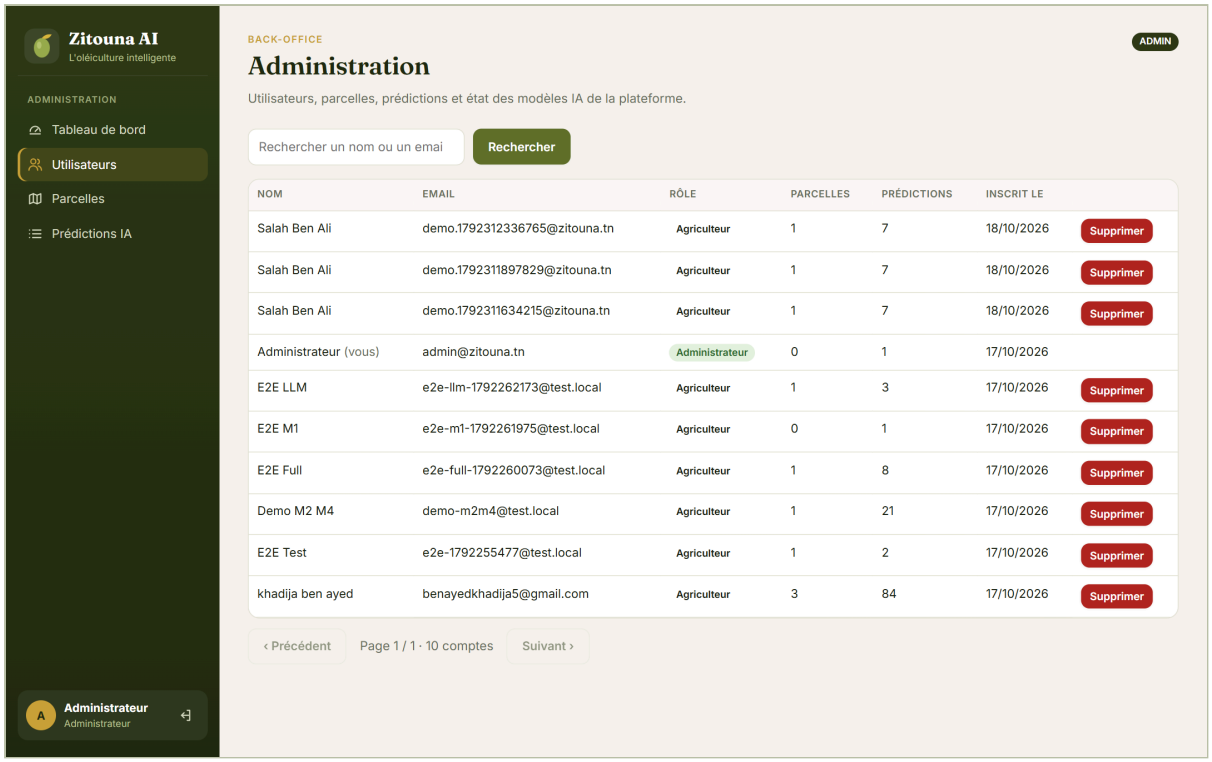


Figure 28 : Back-office : gestion des comptes utilisateurs

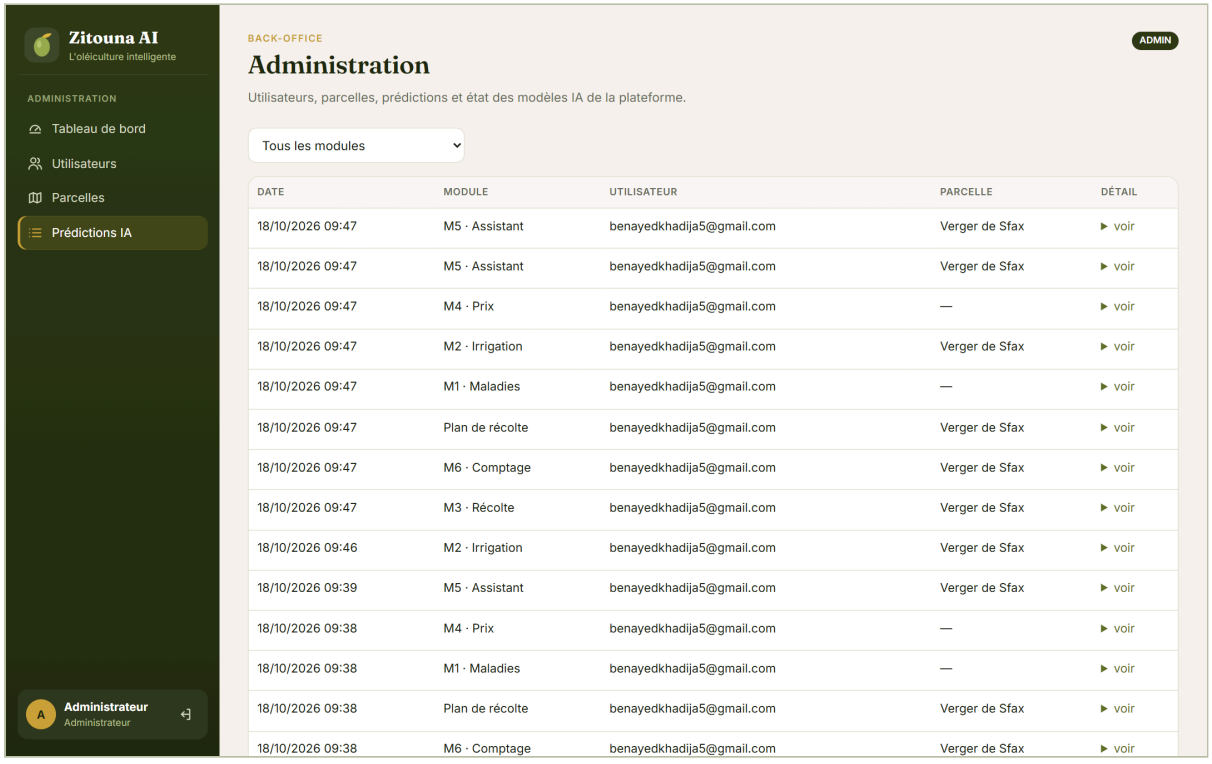


Figure 29 : Back-office : détail de toutes les prédictions IA

5.5 Sécurité

- Mots de passe hachés avec BCrypt, jamais stockés en clair.
- Jeton JWT signé (HS256), valable 24 heures, vérifié à chaque requête.

- Routes `/api/admin/**` réservées au rôle ADMIN : un agriculteur qui les appelle reçoit un refus (403).
- Un seul compte administrateur, défini dans la configuration et vérifié à chaque démarrage ; toute inscription crée un agriculteur.
- Secrets (clé JWT, mot de passe administrateur, clés Gemini et Groq) uniquement dans le fichier `.env`, exclu de Git.

5.6 Tests et qualité

154 tests automatiques vérifient l'ensemble de la plateforme ; tous réussissent.

Partie	Tests	Ce qu'ils vérifient
Backend (Spring Boot)	44	Authentification, parcelles, clients des six modules, back-office, administrateur unique, statut
Frontend (Angular)	15	Irrigation, prix et graphique, accueil, back-office, redirection selon le rôle
M1	7	Contrat de l'API, modèle TorchScript, seuil de confiance
M2	8	LSTM en NumPy, bilan hydrique, alertes, panne de la météo
M3	7	Saison publiée, prévision récursive, surface ou nombre d'arbres
M4	9	Intervalle, conseil vendre / stocker, marge de prudence
M5	55	Règles de clarification, LLM, modèle évalué sur le test indépendant
M6	9	Comptage sur vergers de test, couronnes collées, sol nu

Tableau 17 : Tests automatiques par partie

5.7 Captures des tests

Les captures suivantes ont été prises par l'équipe le 18 octobre 2026, en lançant chaque série de tests depuis PowerShell sur le poste de développement : Maven pour le backend, Vitest pour le frontend et pytest pour chaque service d'IA, dans les mêmes images Docker que l'application. Tous les tests réussissent. Le seul test ignoré (M5) appelle le modèle de langage en direct : il demande une clé d'API et n'est donc lancé qu'à la demande.

```
PS C:\Users\benay\Downloads\ZitounAI-demo-m1\ZitounAI-demo-m1> docker run --rm -v "${PWD}/backend:/src" -v zitounai-m2:/
root/.m2 -w /src eclipse-temurin:21-jdk sh -c 'MVN=$(ls -d /root/.m2/wrapper/dists/apache-maven-*/*/bin/mvn | head -1);
$MVN -B -o verify' 2>$null | Select-String "Tests run|BUILD|Total time"

[INFO] Building zitouna-backend 0.0.1-SNAPSHOT
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 82.22 s -- in tn.zitouna.admin.AdminControllerTes
t
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.250 s -- in tn.zitouna.ai.assistant.AnswerClien
tTest
[INFO] Tests run: 18, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 4.091 s -- in tn.zitouna.ai.assistant.ChatServic
eTest
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.729 s -- in tn.zitouna.ai.assistant.IntentClien
tTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.013 s -- in tn.zitouna.ai.assistant.modules.Ara
bicRepliesTest
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.227 s -- in tn.zitouna.ai.irrigation.Irrigation
AndPriceClientTest
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 28.54 s -- in tn.zitouna.auth.AuthAndParcelFlowTe
st
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.692 s -- in tn.zitouna.status.StatusControllerT
est
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.871 s -- in tn.zitouna.ZitounaBackendApplicatio
nTests
[INFO] Tests run: 44, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
[INFO] Total time:  02:21 min


PS C:\Users\benay\Downloads\ZitounAI-demo-m1\ZitounAI-demo-m1>
```

Figure 30 : Tests du backend Spring Boot (Maven, JUnit 5) : 44 tests réussis

```
PS C:\Users\benay\Downloads\ZitounAI-demo-m1\ZitounAI-demo-m1> cd frontend
PS C:\Users\benay\Downloads\ZitounAI-demo-m1\ZitounAI-demo-m1\frontend> npx ng test --watch=false --reporters=verbose 2>$null | Select-String "s
pec.ts|Test Files|Tests "

 ✓ frontend src/app/shared/price-chart.spec.ts > chartGeometry > draws history, forecast, the bridge between them and a closed band 9ms
 ✓ frontend src/app/shared/price-chart.spec.ts > chartGeometry > keeps every point inside the drawing area, higher prices higher on screen
2ms
 ✓ frontend src/app/shared/price-chart.spec.ts > chartGeometry > handles empty data 0ms
 ✓ frontend src/app/core/auth/auth.interceptor.spec.ts > authInterceptor > sends no Authorization header when logged out 38ms
 ✓ frontend src/app/core/auth/auth.interceptor.spec.ts > authInterceptor > adds the JWT when a valid session exists 7ms
 ✓ frontend src/app/core/auth/auth.interceptor.spec.ts > authInterceptor > ignores an expired session 3ms
 ✓ frontend src/app/app.spec.ts > App > should create the app 55ms
 ✓ frontend src/app/features/price/price.spec.ts > Price > asks 12 weeks by default and shows the advice, the key figures, the chart and
the source 136ms
 ✓ frontend src/app/features/irrigation/irrigation-panel.spec.ts > IrrigationPanel > loads 7 days, shows alerts and totals, reloads 14
days on demand 140ms
 ✓ frontend src/app/features/admin/admin.spec.ts > home after login > sends the admin to the back-office and a farmer to their dashboard
57ms
 ✓ frontend src/app/features/admin/admin.spec.ts > adminGuard > sends farmers back to their dashboard and lets admins in 9ms
 ✓ frontend src/app/features/home/home.spec.ts > Home > greets the farmer, sums the farm and links the tools to the first parcel 138ms
 ✓ frontend src/app/features/assistant/assistant.spec.ts > Assistant > shows suggestion buttons on clarify and resends the question with
the chosen intent 155ms
 ✓ frontend src/app/features/admin/admin.spec.ts > AdminDashboard > shows the totals, the usage per module and the live state of the
models 108ms
 ✓ frontend src/app/features/admin/admin.spec.ts > AdminUsers > lists accounts, deletes a farmer, never offers to delete or change the
single admin 38ms
 Test Files  8 passed (8)
      Tests  15 passed (15)
   Duration  3.80s (transform 1.03s, setup 6.32s, import 1.79s, tests 916ms, environment 16.39s)


PS C:\Users\benay\Downloads\ZitounAI-demo-m1\ZitounAI-demo-m1\frontend>
```

Figure 31 : Tests du frontend Angular (Vitest) : 15 tests réussis

```
PS C:\Users\benay\Downloads\ZitounAI-demo-m1\ZitounAI-demo-m1> docker run --rm -v "${PWD}:/p" -v zitounai-pip:/root/.cache/pip -w /p/ai-services
/m1-disease -e MODEL_DIR=/p/models/m1 zitounai-train-m1 sh -c "pip install -q --root-user-action=ignore --disable-pip-version-check pytest httpx
2 python-multipart && python -m pytest -v -p no:cacheprovider --disable-warnings tests"
============================= test session starts ==============================
platform linux -- Python 3.12.15, pytest-9.1.1, pluggy-1.7.0 -- /usr/local/bin/python
rootdir: /p/ai-services/m1-disease
plugins: platformdirs-4.13.3, anyio-4.15.1
collecting ... collected 7 items

tests/test_api.py::test_health PASSED                                    [ 14%]
tests/test_api.py::test_predict_returns_contract_fields PASSED           [ 28%]
tests/test_api.py::test_rejects_non_image PASSED                         [ 42%]
tests/test_api.py::test_treatments_covers_classes PASSED                 [ 57%]
tests/test_api.py::test_real_model_high_confidence PASSED                [ 71%]
tests/test_api.py::test_real_model_low_confidence_gives_retake_advice PASSED [ 85%]
tests/test_api.py::test_real_model_through_http_endpoint PASSED          [100%]

============================== 7 passed, 16 warnings in 6.57s ==============================
PS C:\Users\benay\Downloads\ZitounAI-demo-m1\ZitounAI-demo-m1>
```

Figure 32 : Tests du service M1 · Maladies (pytest) : 7 tests réussis

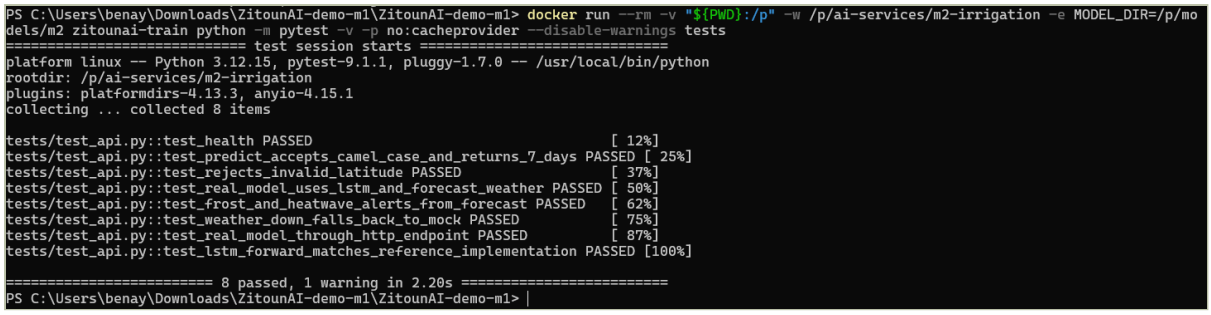


Figure 33 : Tests du service M2 · Irrigation (pytest) : 8 tests réussis

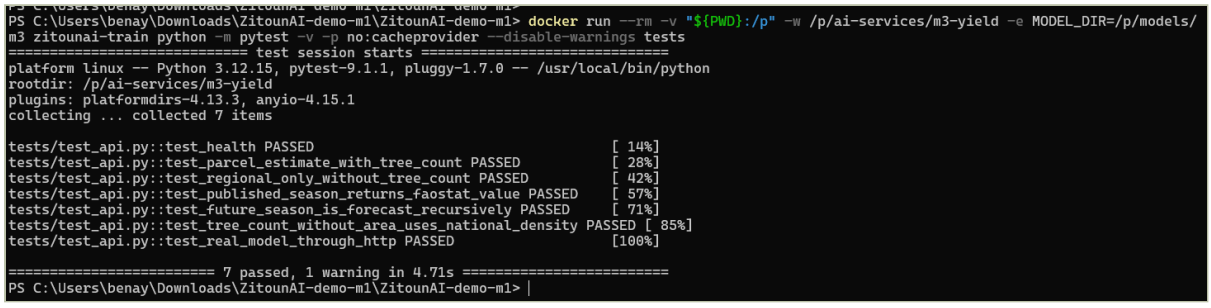


Figure 34 : Tests du service M3 · Récolte (pytest) : 7 tests réussis

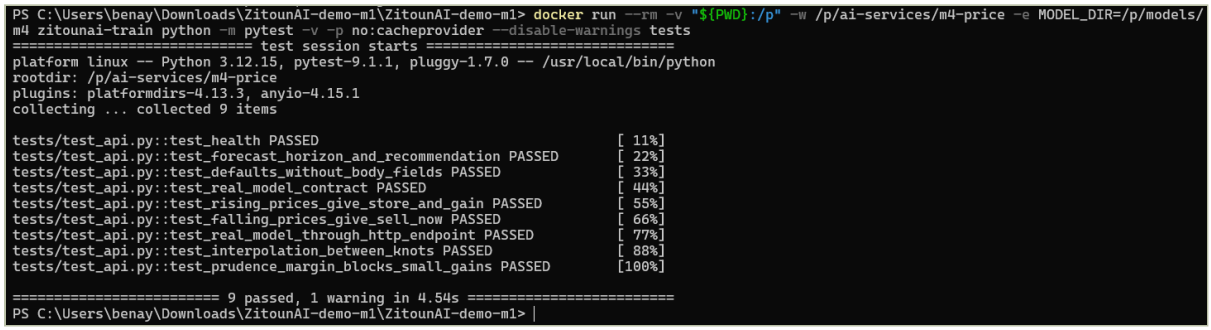


Figure 35 : Tests du service M4 · Prix (pytest) : 9 tests réussis



Figure 36 : Tests du service M5 · Assistant (pytest) : 55 tests réussis, 1 test en direct ignoré

```
PS C:\Users\benay\Downloads\ZitounAI-demo-m1\ZitounAI-demo-m1> docker run --rm -v "${PWD}:/p" -v zitounai-pip:/root/.cache/pip -w /p/ai-services
/m6-tree-count zitounai-train-cv sh -c "pip install -q --root-user-action=ignore --disable-pip-version-check python-multipart && python -m pytes
t -v -p no:cacheprovider --disable-warnings tests"
============================= test session starts ==============================
platform linux -- Python 3.12.15, pytest-9.1.1, pluggy-1.7.0 -- /usr/local/bin/python
rootdir: /p/ai-services/m6-tree-count
plugins: platformdirs-4.13.3, anyio-4.15.1
collecting ... collected 9 items

tests/test_api.py::test_health PASSED                                    [ 11%]
tests/test_api.py::test_detector_is_real_not_mock PASSED                 [ 22%]
tests/test_api.py::test_counts_separate_trees_within_5_percent[4-6-1] PASSED [ 33%]
tests/test_api.py::test_counts_separate_trees_within_5_percent[6-8-2] PASSED [ 44%]
tests/test_api.py::test_counts_separate_trees_within_5_percent[8-10-3] PASSED [ 55%]
tests/test_api.py::test_touching_crowns_are_split PASSED                 [ 66%]
tests/test_api.py::test_bare_soil_has_no_tree PASSED                     [ 77%]
tests/test_api.py::test_http_endpoint_returns_boxes_and_annotated_image PASSED [ 88%]
tests/test_api.py::test_rejects_non_image PASSED                         [100%]

======================== 9 passed, 1 warning in 2.00s ========================
PS C:\Users\benay\Downloads\ZitounAI-demo-m1\ZitounAI-demo-m1>
```

Figure 37 : Tests du service M6 · Comptage (pytest) : 9 tests réussis

5.8 Déploiement

L'ensemble de la plateforme est décrit dans un fichier Docker Compose de neuf conteneurs : la base PostgreSQL, le backend, le frontend et les six services d'IA. Une seule commande, `docker compose up -d`, lance tout sur un PC ; l'application est alors disponible sur http://localhost:4200 et la page d'état du backend sur http://localhost:8080. La procédure complète est donnée en annexe A.

5.9 Conclusion

L'application réunit les six modules dans une interface unique, sécurisée, testée et déployable en une commande, ce qui répond aux besoins fonctionnels et non fonctionnels définis au chapitre 2.

Chapitre 6 : IA générative, qualité et sécurité du logiciel

6.1 Introduction

Au-delà des six modèles intégrés dans l'application, l'IA générative a été utilisée comme outil de génie logiciel tout au long du projet : pour écrire du code, générer et exécuter des tests, détecter des bugs et des vulnérabilités, proposer des améliorations et produire la documentation. Ce chapitre décrit ces usages, les prompts employés, les gains observés, les techniques d'optimisation appliquées aux modèles, et les limites éthiques et techniques du projet.

6.2 Outils d'IA générative utilisés

Outil	Usage dans le projet	Exemples concrets
Claude Code (Anthropic)	Assistant de développement en ligne de commande, connecté au dépôt	Code des services FastAPI, du backend et du frontend ; génération des notebooks ; écriture et exécution des tests ; diagnostic des erreurs Docker et Maven ; revue de sécurité ; rapport, présentation et vidéo
Gemini, Groq (API)	Modèles de langage intégrés dans l'application (M5)	Rédaction des réponses de l'assistant en derja à partir des chiffres des modules
Outils d'analyse classiques	Contrôle des propositions de l'IA	Bandit (sécurité Python), npm audit (dépendances), Ruff (style), JUnit, Vitest et pytest (tests)

Tableau 18 : Outils d'IA générative et outils de contrôle

Principe suivi : chaque proposition de l'IA est vérifiée par une exécution réelle (tests, entraînement, appel de l'API, capture d'écran) avant d'être gardée. Aucun chiffre du rapport ne provient de l'IA sans avoir été mesuré.

6.3 Démarche et exemples de prompts

Les échanges avec l'assistant suivent une boucle courte : une demande en langage naturel, une proposition de l'IA, une exécution, puis une correction si le résultat ne convient pas. Le tableau suivant reprend des prompts réels du journal d'interactions de l'intégration (Khadija Ben Ayed) ; les fiches de chaque membre figurent en annexe C.

Prompt (extrait réel)	Action de l'IA	Vérification humaine
« vérifier le projet run ou non c'est correct ? »	Lancement des 9 conteneurs, exécution des tests backend et IA un par un dans Docker, rapport des échecs	Logs Docker et résultats des tests relus
« je veux avec API non mock des valeurs réelles »	Remplacement des valeurs simulées par Open-Meteo (M2) et SARIMA sur les prix FMI (M4), avec repli explicite si l'API tombe	Appel réel de l'API, comparaison avec la prévision Open-Meteo

« je veux éliminer swagger complètement … un projet réel pas de mock »	Suppression de springdoc et de la configuration OpenAPI, ajout d'une page d'état, entraînement réel de M1 (Kaggle) et de M6 (OpenCV)	Page /api/status : 6 « Modèle réel »
« je veux un compte admin et un compte utilisateur … admin il a 1 seule compte »	Administrateur unique créé depuis la configuration, routes /api/admin protégées, redirection selon le rôle	Test : agriculteur → 403, administrateur → 200
« tous les modules run correctement dans la vidéo ? »	Lecture des prédictions enregistrées en base pendant la vidéo (champ mock) et extraction d'une image par scène	7 appels réels, aucun en mode de repli
« compare avec la grille d'évaluation »	Analyse des critères, revue de sécurité (Bandit, npm audit, relecture du code), corrections et ce chapitre	Tests relancés après chaque correction

Tableau 19 : Exemples de prompts et de leur vérification

6.4 Gains de productivité observés

Tâche	Apport de l'IA	Limite constatée
Code répétitif (DTO, contrôleurs, composants)	Génération rapide en suivant les conventions du dépôt	Relecture nécessaire pour la cohérence d'ensemble
Tests automatiques	Écriture des cas limites (vergers synthétiques pour M6, jeu de questions sondes pour M5, rôles pour le back-office)	Un test peut être ignoré en silence si l'environnement est faux (cas M5, section 6.5)
Diagnostic d'erreurs	Analyse des journaux Docker, Maven et Word, et proposition de contournements	Plusieurs essais parfois nécessaires
Documentation	Rapport, présentation, captures et vidéo produits à partir du code et des résultats réels	Contenu à valider par l'équipe

Tableau 20 : Gains et limites de l'IA générative dans le développement

Le principal gain est la vitesse d'itération : une idée est codée, testée et corrigée dans la même séance. La contrepartie est la vigilance : l'IA s'est trompée à plusieurs reprises (commandes incompatibles avec Windows, hypothèses fausses sur l'environnement), ce qui confirme qu'elle accélère le travail mais ne remplace pas la vérification.

6.5 Détection de bugs et tests assistés par l'IA

Après l'intégration, l'assistant a lancé les tests, lu les échecs et proposé des corrections. Le tableau suivant liste des défauts réels détectés ainsi.

Partie	Défaut détecté	Correction
M3	Entraînement sur des saisons sans production connue (valeurs manquantes)	Ajustement limité aux saisons publiées (jusqu'à 2024)
M4	Le conseil « stocker » perdait de l'argent sur le test (−0,47 TND/kg)	Marge de prudence de 4 % choisie sur la validation : +0,10 TND/kg
M5	Clarification demandée trop souvent avec un seuil trop strict	Seuil de 0,35 choisi pour garder 85 % de justesse

M5	Le modèle gpt-oss-120b inventait des chiffres et mélangeait les écritures	Modèle retiré de la chaîne de LLM
M5	Tests du modèle ignorés en silence quand le dossier du modèle était mal configuré	Chemin du modèle fixé explicitement ; recommandation : faire échouer ce cas en intégration continue
Backend	Deux tests partageaient la même base et créaient le même e-mail	E-mails uniques par test
Frontend	Le séparateur de milliers français (espace insécable fine) cassait une assertion	Assertion indépendante du type d'espace
Frontend	La liste déroulante n'affichait pas l'option choisie	Sélection portée par chaque option

Tableau 21 : Défauts détectés avec l'assistance de l'IA

6.6 Revue de sécurité

Une revue de sécurité a été menée le 18 octobre 2026 en combinant des outils d'analyse et une relecture du code par l'IA.

- **Bandit** (code Python des six services) : 2 alertes de gravité moyenne. M1 chargeait les poids du modèle avec `torch.load` sans restriction (exécution de code possible depuis un fichier piégé, CWE-502) ; M2 appelait `urlopen` avec une URL configurable qui accepterait `file://` (CWE-22).
- **Corrections** : `weights_only=True` pour M1, et URL météo limitée à `https://` pour M2. Les tests de M1 (7) et de M2 (8) passent toujours, et Bandit ne signale plus aucun problème.
- **npm audit** (dépendances du frontend) : aucune vulnérabilité.
- **Recherche de secrets** dans le dépôt : aucune clé d'API ni mot de passe en clair ; le fichier `.env` est exclu de Git.
- **Relecture du backend** : jeton JWT HS256 avec une clé d'au moins 32 caractères imposée au démarrage, BCrypt, CORS limité au frontend, mots de passe de 8 à 72 caractères, envois limités à 10 Mo.

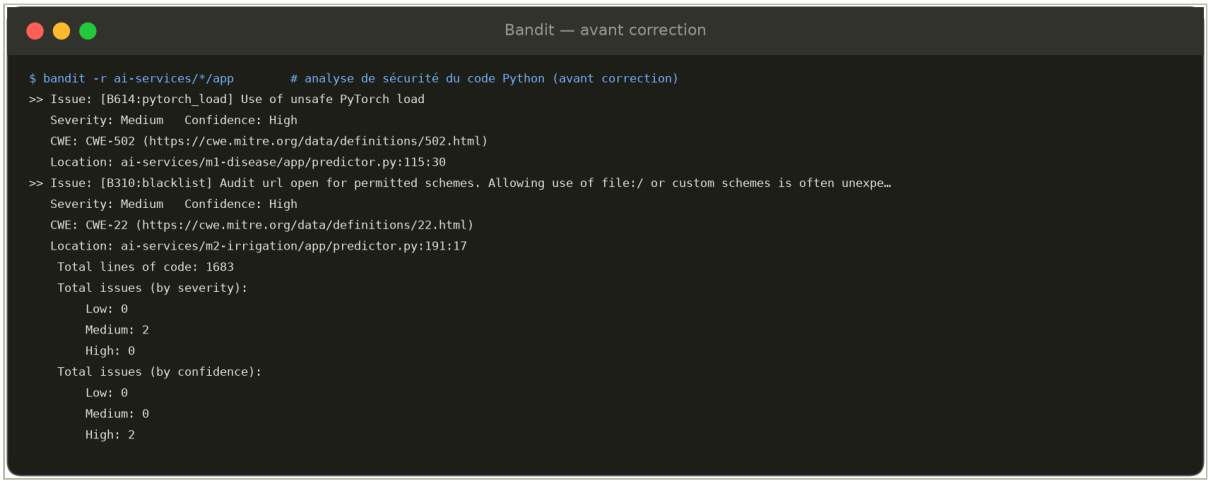


Figure 38 : Analyse Bandit avant correction : deux alertes de gravité moyenne



Figure 39 : Bandit après correction et npm audit : aucun problème

6.7 Recommandations de refactoring

Recommandation	Justification
Limiter le nombre de tentatives de connexion	La route de connexion n'a pas de limite de débit : risque d'attaque par force brute
Supprimer la clé JWT par défaut	Une clé de développement est utilisée si `JWT_SECRET` manque ; mieux vaut refuser de démarrer
Jetons courts avec renouvellement	Un jeton de 24 heures ne peut pas être révoqué en cas de vol
Faire échouer les tests si un modèle manque en intégration continue	Évite qu'un test ignoré passe inaperçu (cas M5)
Alléger l'image du service M5 (2,4 Go)	PyTorch n'y sert plus depuis que le classifieur TF-IDF suffit
Mettre en commun le chargement des modèles des six services	Le même schéma (dossier, version, repli) est répété dans chaque service

Tableau 22 : Recommandations de refactoring issues de la revue

6.8 Adaptation et optimisation des modèles

Technique	Modules	Mise en œuvre
Choix et comparaison de modèles	Tous	Chaque modèle servi est comparé à une référence simple et à d'autres modèles sur des données de test
Fine-tuning	M1	MobileNetV3 préentraîné sur ImageNet, affiné en deux phases sur les photos de feuilles
Réglage des hyperparamètres et des seuils	M1, M2, M4, M5	Température de calibration (0,49) et seuil de 70 % (M1), fenêtre de 30 jours (M2), ordre SARIMA et marge de 4 % (M4), seuil de clarification de 0,35 (M5), tous choisis sur la validation
Prompt engineering	M5	Prompt système de 10 règles (derja en lettres arabes, 60 mots maximum, aucun chiffre inventé, prudence sur les traitements) et exemples few-shot
Génération ancrée dans les données (approche de type RAG)	M5	Les chiffres des modules M2, M3 et M4 sont injectés dans le prompt : le LLM reformule des faits au lieu de les inventer

Protection contre l'injection de prompt	M5	Règle explicite : ignorer toute instruction de l'utilisateur contraire aux règles
Réduction du coût, de la latence et des tokens	M5	Raisonnement désactivé sur Gemini, 700 tokens maximum, température 0,3, délai de 5 s par appel et 12 s au total, chaîne de 6 modèles gratuits avec mise en pause d'un modèle en échec
Cache	M2, M4	Météo mise en cache 1 heure par zone de 10 km (M2) ; SARIMA ajusté une fois au démarrage (M4)
Optimisation de l'inférence	M1, M2, M6	TorchScript (M1), LSTM exécuté en NumPy sans PyTorch (M2), détection OpenCV sans GPU (M6)

Tableau 23 : Techniques d'adaptation et d'optimisation appliquées

6.9 Limites éthiques et techniques

- **Données personnelles** : minimisation des données envoyées aux LLM (ni nom ni e-mail), mots de passe hachés, secrets hors du dépôt.
- **Biais des données** : photos de feuilles prises sur fond uni (M1), questions écrites par l'équipe (M5), production nationale et non régionale (M3), prix mondial et non prix au producteur (M4).
- **Hallucinations** : le LLM ne reçoit que des faits mesurés, ne donne jamais de dose de produit et conseille de consulter un technicien avant tout traitement.
- **Responsabilité** : l'application conseille, l'agriculteur décide ; les intervalles de confiance, la carte Grad-CAM et les sources sont affichés pour qu'il juge par lui-même.
- **Dépendance externe** : les LLM et la météo sont des services tiers soumis à des quotas ; des solutions de repli maintiennent une réponse en cas de panne.
- **Sobriété** : modèles légers exécutés sans carte graphique.

6.10 Conclusion

L'IA générative a accéléré chaque étape du projet, du code aux tests et à la documentation, à condition de vérifier chacune de ses propositions par une exécution réelle. La revue de sécurité assistée par l'IA a permis de corriger deux vulnérabilités et de dresser une liste d'améliorations concrètes.

Conclusion générale et perspectives

Ce projet avait pour but d'aider le petit oléiculteur tunisien à prendre de meilleures décisions grâce à l'intelligence artificielle. Zitouna AI atteint le critère de réussite fixé au départ : les six modules fonctionnent ensemble dans une même application, chacun avec un modèle entraîné sur des données réelles et une métrique qui bat une référence simple.

Le projet couvre toute la chaîne d'un produit d'IA : collecte des données, modélisation, évaluation, microservices, backend sécurisé, interface agriculteur, back-office et déploiement Docker, vérifiés par 154 tests automatiques. Il contribue aussi à plusieurs objectifs de développement durable, en particulier une agriculture plus productive (ODD 2), une meilleure gestion de l'eau (ODD 6) et une meilleure résilience face au climat (ODD 13).

Ce travail nous a appris trois leçons. D'abord, il faut toujours commencer par une référence simple : elle montre si un modèle complexe apporte vraiment quelque chose (le LSTM de M4 ne bat pas la prévision naïve, SARIMA oui). Ensuite, un découpage des données sans fuite donne des chiffres honnêtes, même s'ils sont moins flatteurs. Enfin, un bon modèle ne suffit pas : la marge de prudence de M4 et le seuil de confiance de M1 sont ce qui rend les conseils vraiment fiables pour l'agriculteur.

Plusieurs perspectives prolongent ce travail :

Limite actuelle	Perspective
M3 prévoit la production nationale, faute de données régionales ouvertes	Extraire les rapports ONAGRI par gouvernorat et entraîner le même pipeline par région
M4 utilise un prix mondial de référence	Saisir les prix au producteur tunisiens du Conseil oléicole international
M5 repose sur 500 questions et un modèle TF-IDF	Enrichir les questions et comparer un modèle SetFit ou BERT entraîné sur GPU
M6 utilise une méthode classique évaluée sur des images générées	Entraîner YOLOv8 sur OliveTreeCrownsDb et l'évaluer sur des images tunisiennes annotées
Application web uniquement	Version mobile, utilisable au champ, et tests avec de vrais agriculteurs

Tableau 24 : Limites et perspectives

Bibliographie et webographie

1. Uğuz S., Uysal N. *Olive Leaf Image Dataset*, Kaggle. https://www.kaggle.com/
2. Open-Meteo. *Historical Weather API (ERA5) et Forecast API*. https://open-meteo.com/
3. FAO. *FAOSTAT, statistiques de production agricole*. https://www.fao.org/faostat/
4. Fonds monétaire international, via FRED. *Global price of Olive Oil (POLVOILUSDM)*. https://fred.stlouisfed.org/series/POLVOILUSDM
5. Banque mondiale. *Taux de change officiel (PA.NUS.FCRF), Tunisie*. https://data.worldbank.org/indicator/PA.NUS.FCRF?locations=TN
6. Allen R. G., Pereira L. S., Raes D., Smith M. *Crop evapotranspiration, FAO Irrigation and Drainage Paper 56*, FAO, 1998.
7. Howard A. et al. *Searching for MobileNetV3*, ICCV, 2019.
8. Selvaraju R. R. et al. *Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization*, ICCV, 2017.
9. Hochreiter S., Schmidhuber J. *Long Short-Term Memory*, Neural Computation, 1997.
10. Breiman L. *Random Forests*, Machine Learning, 2001.
11. Otsu N. *A Threshold Selection Method from Gray-Level Histograms*, IEEE Transactions on Systems, Man, and Cybernetics, 1979.
12. Nations unies. *Objectifs de développement durable*. https://www.un.org/sustainabledevelopment/fr/
13. ONAGRI, Observatoire national de l'agriculture. Chiffres du secteur oléicole cités dans le plan de projet.
14. Documentations officielles : Spring Boot, Angular, FastAPI, PyTorch, scikit-learn, statsmodels, OpenCV, Docker.

Annexes

Annexe A : Lancer l'application

1. Installer Docker Desktop sur le PC.
2. Copier le fichier `.env.example` en `.env`, puis remplir `JWT_SECRET`, `ADMIN_EMAIL`, `ADMIN_PASSWORD` et, pour l'assistant, `GEMINI_API_KEY` et `GROQ_API_KEY`.
3. Placer les modèles entraînés dans le dossier `models/` (ils ne sont pas dans Git).
4. À la racine du projet, lancer `docker compose up -d` ; le premier lancement construit les images.
5. Ouvrir l'application sur http://localhost:4200 et la page d'état du backend sur http://localhost:8080.
6. Se connecter avec le compte administrateur défini dans `.env`, ou créer un compte agriculteur avec « Créer un compte ».

Annexe B : Glossaire

Terme	Signification
Calibration	Accord entre la confiance affichée par un modèle et son taux réel de bonnes réponses
Derja, arabizi	Arabe tunisien ; l'arabizi l'écrit en lettres latines et en chiffres (9 pour ق)
F1 macro	Moyenne du score F1 de chaque classe ; ne favorise pas la classe la plus fréquente
Grad-CAM	Carte qui montre la zone de l'image qui a compté dans la décision du réseau
Microservice	Petit service indépendant, avec sa propre API, déployé dans son conteneur
Précision (accuracy)	Part des prédictions justes
Référence (baseline)	Méthode simple qu'un vrai modèle doit battre pour être utile
Seuil d'Otsu	Méthode qui choisit automatiquement le seuil séparant deux groupes de pixels
Validation glissante	Évaluation où chaque période est prévue par un modèle entraîné sur les seules périodes précédentes

Annexe C : Documentation individuelle

Chaque membre documente son usage de l'IA générative sur une fiche au format ci-dessous : les prompts envoyés aux LLM, ce qui a été gardé ou corrigé, et la justification du modèle d'IA choisi pour son module. La fiche de Khadija Ben Ayed est reproduite à titre d'exemple ; les fiches des autres membres sont jointes au rapport.

C.1 Modèle de fiche

Rubrique	Contenu attendu
Membre et module	Nom, module d'IA porté, rôle transversal
Outils d'IA générative	Claude, Copilot, ChatGPT… et pour quelles tâches

Journal des prompts	Pour chaque échange important : le prompt, la réponse ou l'action de l'IA, ce qui a été gardé ou modifié, la vérification faite
Justification du modèle choisi	Modèles comparés, métrique, raison du choix (performance, coût, rapidité, explicabilité)
Gain de productivité	Tâches accélérées, erreurs de l'IA corrigées, limites rencontrées

Tableau 25 : Modèle de fiche de documentation individuelle

C.2 Fiche de Khadija Ben Ayed (M2 · Irrigation, intégration backend)

Rubrique	Contenu
Outils d'IA générative	Claude Code : intégration des modules, backend, tests, revue de sécurité, rapport, présentation et vidéo de démonstration
Prompt	« tu peux le travailler M2 et M4, je veux un travail correct, je veux le valider demain »
Action de l'IA et vérification	Entraînement du LSTM sur ERA5, service FastAPI, écran d'irrigation ; vérifié par 8 tests et une comparaison avec la prévision d'Open-Meteo à Sfax
Prompt	« je veux avec API non mock des valeurs réelles »
Action de l'IA et vérification	Branchement de l'API Open-Meteo avec cache et repli explicite ; vérifié par un appel réel et le test « panne de la météo »
Prompt	« tous les modules run correctement dans la vidéo ? »
Action de l'IA et vérification	Contrôle des prédictions enregistrées en base pendant la vidéo : 7 appels réels, aucun en mode de repli
Justification du modèle M2	Comparé à « même jour l'an dernier » (1,06 mm/j), à la persistance (0,80) et à Prophet : le LSTM a la plus faible erreur (0,71 mm/j) et s'exécute en NumPy, ce qui allège le service et le rend rapide
Gain de productivité	Intégration et tests produits dans la même séance ; erreurs de l'IA corrigées (commandes Windows, chemins de modèles), chaque résultat vérifié par exécution

Tableau 26 : Fiche de documentation individuelle de Khadija Ben Ayed